

COMPREHENSIVE EE & ECE FORMULA & REFERENCE CHEAT SHEET

*Electronics • BJT • MOSFET • SCR • Op-Amp • Filters • Digital • Analogue • Embedded • MCU • Memory • Industrial
Comms • Metering • WebSockets • REST API • PoE*

Final Year / Professional Engineering Reference | 17 Comprehensive Sections

1. SEMICONDUCTOR DIODES

DIODE FUNDAMENTALS			
Shockley Eqn	$I = I_0(e^{(V/nVT)} - 1)$	Thermal Voltage	$V_T = kT/q \approx 26 \text{ mV @ } 300\text{K}$
Ideality Factor n	1 (ideal), 1-2 (real)	Forward V (Si)	$V_F \approx 0.7 \text{ V}$
Forward V (Ge)	$V_F \approx 0.3 \text{ V}$	Forward V (Schottky)	$V_F \approx 0.2\text{--}0.4 \text{ V}$
Reverse Sat Current	$I_0 \approx 10^{-12} \text{ A (Si)}$	Knee Voltage	$V_Y \approx 0.5 \text{ V (Si)}$
RECTIFIER CIRCUITS			
Half-Wave Vdc	$V_{dc} = V_m / \pi$	HW Ripple Factor	$\gamma = 1.21$
Full-Wave Vdc	$V_{dc} = 2V_m / \pi$	FW Ripple Factor	$\gamma = 0.482$
Bridge Vdc	$V_{dc} = 2V_m / \pi$	Bridge PIV	$PIV = V_m$
Ripple Factor	$\gamma = V_{rms(ac)} / V_{dc}$	Capacitor Filter C	$C = I_L / (2fV_r)$
ZENER & SPECIAL DIODES			
Zener Regulation	$V_{out} = V_Z \text{ (constant)}$	Zener Resistance	$r_z = \Delta V_Z / \Delta I_Z$
Zener < 5V	Tunneling (Zener) breakdown	Zener > 5V	Avalanche breakdown
LED Wavelength	$\lambda = hc / E_g \text{ (nm)}$	Photodiode	$I_L = S \times \Phi \text{ (A/W} \times \text{W)}$
Varactor Cap	$C_j = C_0 / (1 + V_R/V_0)^n$	Schottky	Metal-semiconductor junction

2. BJT TRANSISTORS (Bipolar Junction)

BJT BASICS & CURRENT RELATIONS			
Current gain β (hFE)	$\beta = I_C / I_B$	Alpha α	$\alpha = I_C / I_E = \beta / (\beta + 1)$
KCL	$I_E = I_C + I_B$	β from α	$\beta = \alpha / (1 - \alpha)$
IC (active)	$I_C = \beta \times I_B + (1 + \beta) I_{CBO}$	ICEO	$I_{CEO} = (1 + \beta) \times I_{CBO}$
OPERATING REGIONS			
Cut-off	$V_{BE} < 0.5\text{V}$ (switch OFF)	Active (Amplify)	$V_{BE} \approx 0.7\text{V}, V_{CE} > V_{CE(sat)}$
Saturation (ON)	$V_{CE(sat)} \approx 0.2\text{V}$, both junctions fwd	Reverse Active	BE reverse, BC forward (rare)
SMALL-SIGNAL MODEL (h-parameters)			
Transconductance g_m	$g_m = I_C / V_T = I_C / 0.026$	Input resistance r_{π}	$r_{\pi} = \beta / g_m$
Output resistance r_o	$r_o = V_A / I_C$	Early Voltage V_A	$V_A \approx 50\text{--}300 \text{ V}$
CONFIGURATIONS			
CE Voltage Gain	$A_v = -g_m \times R_C \parallel r_o$	CE Input Z	$Z_{in} = R_B \parallel r_{\pi}$
CE Output Z	$Z_{out} \approx R_C \parallel r_o$	CE Current Gain	$A_I = -\beta$ (phase inv)
CB Voltage Gain	$A_v \approx g_m \times R_C (>0)$	CB Current Gain	$A_I = \alpha < 1$
CB Input Z	$Z_{in} = 1/g_m$ (low)	CB Output Z	$Z_{out} \approx R_C$ (high)
CC (Emitter Flwr)	$A_v \approx 1$ (no phase inv)	CC Input Z	$Z_{in} = \beta \times R_E$ (high)
CC Output Z	$Z_{out} \approx 1/g_m$ (low, $\sim 25\Omega$)	CC Current Gain	$A_I = \beta + 1$
BIASING			
Fixed Bias IB	$I_B = (V_{CC} - V_{BE}) / R_B$	Self-Bias ICQ	$I_{CQ} = (V_{CC} - V_{BE}) / (R_B/\beta + R_E)$
Voltage Divider VB	$V_B = V_{CC} \times R_2 / (R_1 + R_2)$	Q-point stability	$S = \Delta I_C / \Delta I_{CO}$
Thermal Runaway	High $I_C \rightarrow \text{Temp} \uparrow \rightarrow I_C \uparrow$ (instable)	Stability Factor S	$S = (1 + \beta) / (1 - \beta \times dI_B/dI_C)$

POWER & SWITCHING			
Power Dissipation	$PD = V_{CE} \times I_C$	Max Power Hyperbola	$V_{CE} \times I_C = PD(max)$
Switching ton	$t_{on} = t_r + t_d$	Switching toff	$t_{off} = t_s + t_f$
NOTE: Always check β range; real β varies 2:1 across samples. Use voltage-divider bias for stability.			

BJT LOGIC-LEVEL SWITCHING — DUAL RESISTOR NETWORK			
WHY TWO RESISTORS? RB (base series) + R_BE (base-to-emitter pull-down)			
RB — base resistor	Limits base current: $RB = (V_{logic} - V_{BE}) / IB_{min}$	Typical RB	1kΩ – 10kΩ for 3.3V/5V logic
R_BE — pull-down	Pulls base to GND when driver is Hi-Z / off	Typical R_BE	10kΩ – 47kΩ (10× RB recommended)
IB required	$IB = IC / \beta_{min}$ (use worst-case β)	Overdrive x10	$IB = 10 \times IC / \beta$ for hard saturation
VBE threshold	NPN: $V_{BE} \approx 0.7V$ PNP: $V_{EB} \approx 0.7V$	Logic low leak	R_BE holds base $< 0.5V \rightarrow$ device stays OFF
NPN SWITCH (LOW-SIDE) — CIRCUIT & CALCULATIONS			
Circuit (NPN)	VCC—RLOAD—Collector Base—RB—GPIO(3.3V) R_BE: Base—GND Emitter—GND		
RB formula	$RB = (V_{gpio} - V_{BE}) / IB = (3.3V - 0.7V) / (IC / \beta_{min}) \quad [\Omega]$		
R_BE formula	$R_{BE} = 10 \times RB$ (prevents false turn-on from leakage / noise)		
Example 100mA	$IC=100mA, \beta=100, V_{gpio}=3.3V \rightarrow IB=1mA \rightarrow RB=(2.6V/1mA)=2.6k\Omega \rightarrow$ use 2.7kΩ; $R_{BE}=27k\Omega$		
Saturation check	$V_{CE(sat)}=0.2V$ must be $< V_{CC}-V_{RLOAD}$ $IC(sat) = (V_{CC}-0.2V) / R_{LOAD}$		
PNP SWITCH (HIGH-SIDE) — CIRCUIT & CALCULATIONS			
Circuit (PNP)	VCC—Emitter Base—RB—GPIO R_BE: Base—VCC(via pull-up) Collector—RLOAD—GND		
PNP ON condition	GPIO = LOW (0V) $\rightarrow V_{EB} = V_{CC} - 0 > 0.7V \rightarrow$ transistor conducts		
PNP OFF	GPIO = HIGH (VCC) $\rightarrow V_{EB} \approx 0V \rightarrow$ transistor OFF		
RB (PNP)	$RB = (V_{CC} - V_{gpio_low} - V_{EB}) / IB = (V_{CC} - V_{EB}) / IB$ when GPIO pulls to GND		
SWITCHING SPEED CONSIDERATIONS			
Turn-ON delay td	Time for base charge to build (ns-μs)	Turn-OFF ts	Storage time: base charge must clear
Speedup capacitor	C parallel with RB — boosts base drive on edge	Value Cspeed	$C \approx 10\text{--}100pF$ typical (trial with scope)
Baker clamp	Schottky diode: base-collector prevents deep sat.	Benefit	Eliminates storage time ts $\rightarrow$ fast switch
NOTE: Always use R_BE pull-down with GPIO-driven BJT switches. Without it, floating base or leakage current can partially turn on the transistor. Ratio rule: $R_{BE} = 10\times RB$.			

3. MOSFET & FET

MOSFET REGIONS & EQUATIONS			
Enhancement NMOS ON	$V_{GS} > V_{th}$ (typically 1-4V)	Depletion NMOS ON	$V_{GS} = 0$ (normally ON)
Triode (Linear) ID	$I_D = k[(V_{GS}-V_{th})V_{DS} - V_{DS}^2/2]$	Saturation ID	$I_D = (k_n/2)(V_{GS} - V_{th})^2$
Process Transcond. k	$k = \mu_n \times C_{ox} \times W/L$	Cox	$C_{ox} = \epsilon_{ox} / t_{ox}$
Transconductance gm	$g_m = 2I_D / (V_{GS}-V_{th}) = \sqrt{2k \times I_D}$	Output cond. gds	$g_{ds} = I_D / V_A$
Voltage Gain (sat)	$A_v = -g_m \times R_D$	Drain-Source sat.	$V_{DS(sat)} = V_{GS} - V_{th}$
JFET			

Shockley (JFET)	$ID = IDSS(1 - VGS/VP)^2$	Pinch-off VP	$VP < 0$ for N-JFET
gm (JFET)	$gm = -2IDSS/VP \times (1 - VGS/VP)$	IDSS	ID at $VGS = 0$
CMOS & LOGIC			
CMOS Inverter	NMOS pulls down, PMOS pulls up	Static Power	$P \approx 0$ (only leakage)
Dynamic Power	$P = \alpha \times C \times VDD^2 \times f$	Propagation Delay	$tpd = 0.69RC$
CMOS Noise Margin	$NMH = VOH - VIH$	NML	$NML = VIL - VOL$
POWER MOSFET (switching)			
RDS(on)	On-resistance (mΩ range)	Gate Charge QG	$QG = IG \times tg$
Switching Loss	$Psw = 0.5 \times VDS \times ID \times (tr+tf) \times f$	Conduction Loss	$Pcond = ID^2 \times RDS(on)$
Gate Drive Vgs	$Vgs(th) + 10V$ typ for full ON	Safe Operating Area	Check SOA curve in datasheet
NOTE: Higher W/L → lower Vth, higher gm. Use PMOS for high-side, NMOS for low-side switching.			

MOSFET LOGIC-LEVEL SWITCHING — GATE RESISTOR NETWORK			
WHY TWO RESISTORS? RG (gate series) + RGS (gate-to-source pull-down)			
RG — gate resistor	Damps ringing on gate trace (RC with Ciss)	Typical RG	10Ω - 100Ω for discrete MOSFETs
RGS — pull-down	Holds gate at 0V when driver Hi-Z / power-up	Typical RGS	10kΩ - 100kΩ (must not load driver)
Gate threshold Vth	VGS(th): 1-4V logic-level 2-6V standard	Drive Vgs	Logic-level MOSFET: fully ON at 5V
RGS prevents	Floating gate → dV/dt turn-on / ESD latch-up	Gate capacitor	Ciss = Cgd + Cgs (pF-nF range)
N-CHANNEL LOW-SIDE SWITCH — STANDARD CONFIGURATION			
Circuit (NMOS)	VDD—RLOAD—Drain Gate—RG—GPIO RGS: Gate—Source(GND) Source—GND		
Turn-ON time	t_on ≈ RG × Ciss (time constant for gate to charge through RG)		
Turn-OFF time	t_off ≈ (RG + Rdriver_pull-down) × Ciss		
Gate current pk	Ig_peak = (Vdriver - Vgs_plateau) / RG [A] (occurs during Miller plateau)		
Example	Vgpio=3.3V, Ciss=1nF, RG=47Ω → t_on≈47ns; RGS=100kΩ holds gate<0.1V when OFF		
P-CHANNEL HIGH-SIDE SWITCH			
Circuit (PMOS)	VDD—Source Gate—RG—GPIO RGS: Gate—Source(VDD via pull-up) Drain—RLOAD—GND		
PMOS ON	GPIO=LOW → VGS = 0 - VDD = -VDD → VGS > Vth → PMOS conducts		
PMOS OFF	GPIO=HIGH (VDD) → VGS = VDD - VDD = 0V → PMOS OFF		
RGS (PMOS)	Pull-up resistor: Gate—RGS—VDD (holds gate=VDD=OFF when GPIO Hi-Z)		
GATE DRIVER IC (recommended for >1A switching)			
Gate driver function	Provides high peak current (1-10A) for fast switching	Example ICs	UCC27524, TC4427, IR2101, IRS2003
Bootstrap (half-brdg)	High-side NMOS gate drive above VDD via bootstrap C	Dead time	Both switches OFF briefly to prevent shoot-through
Shoot-through	Both high-side & low-side ON simultaneously → short!	Protection	Dead-time + cross-conduction interlock
PARASITIC EFFECTS & MITIGATION			
Miller effect	Cdg amplifies effective input capacitance	dV/dt turn-on	Fast drain edge charges Cgd → raises Vgs
Gate ringing	L_trace × Ciss resonance → overshoot/EMI	Fix	Increase RG or add ferrite bead on gate
Negative VGS spike	Source inductance creates -VGS on turn-off	Fix	Minimize source path inductance; use Kelvin source
NOTE: RGS is mandatory — never leave a MOSFET gate floating. For fast switching: use logic-level FET (Vth <2V) with gate driver IC. Increase RG to reduce EMI at the cost of switching loss.			

4. SCR & THYRISTORS

SCR BASICS			
Structure	PNPN (4-layer) – Anode, Gate, Cathode	Turn-ON	Gate pulse ($I_G > I_{GT}$) while $V_{AK} > 0$
Holding Current I_H	Min I_A to stay latched (gate removed)	Latching Current I_L	Min I_A to turn ON after gate pulse
Turn-OFF	Reduce $I_A < I_H$ (commutation)	Forward Breakover	V_{BO} : turns ON without gate (avoid)
2-transistor model	PNP + NPN regenerative pair	$\alpha_1 + \alpha_2 = 1$	Condition for latching
FIRING ANGLE & POWER CONTROL			
Half-wave avg V_{dc}	$V_{dc} = (V_m/2\pi)(1+\cos\alpha)$	Full-wave avg V_{dc}	$V_{dc} = (V_m/\pi)(1+\cos\alpha)$
Firing angle α	Delay from voltage zero-crossing ($^\circ$)	Conduction angle	$\gamma = \pi - \alpha$ (half-wave)
Power delivered	$P = V_{rms}^2 / R$ (V_{rms} varies with α)	RMS output	$V_{rms} = V_m \sqrt{[(\pi - \alpha + \sin 2\alpha/2)/2\pi]}$
TRIAC & DIAC			
TRIAC	Bidirectional SCR (AC control)	DIAC	Bilateral trigger diode (breakover $\sim 32V$)
TRIAC trigger	Gate pulse in any quadrant (Q1-Q4)	Snubber RC	Prevents dV/dt false triggering
COMMUTATION METHODS			
Natural (line)	AC supply reversal turns OFF SCR	Forced	External LC circuit forces $I_A=0$
Class A	Self commutated by LC resonance	Class B	Separate commutation capacitor
NOTE: SCRs cannot be turned OFF by gate. Use IGBT or power MOSFET if active turn-off is needed.			

5. OPERATIONAL AMPLIFIERS

IDEAL OP-AMP RULES & SPECS			
Virtual Short	$V_+ = V_-$ (with neg. feedback)	No Input Current	$I_+ = I_- = 0$
Ideal AOL	$AOL = \infty$	Ideal Z_{in}	$Z_{in} = \infty$
Ideal Z_{out}	$Z_{out} = 0$	Ideal BW	$BW = \infty$, $CMRR = \infty$
GBW Product	$GBW = A_v \times BW = \text{constant}$	Slew Rate SR	$SR = dV_{out}/dt _{max}$ [V/ μs]
CMRR	$CMRR = 20\log(A_d/A_{cm})$ [dB]	PSRR	$20\log(\Delta V_s/\Delta V_{out})$ [dB]
Input Offset V_{os}	Differential error voltage	Bias Current I_B	$I_B = (I_+ + I_-)/2$
AMPLIFIER CONFIGURATIONS			
Inverting Amp	$A_v = -R_f / R_{in}$	Non-Inverting Amp	$A_v = 1 + R_f/R_1$
Voltage Follower	$A_v = 1$, $Z_{in} = \infty$	Difference Amp	$V_{out} = (R_2/R_1)(V_2 - V_1)$
Summing Amp	$V_{out} = -R_f(V_1/R_1 + V_2/R_2 + \dots)$	Instrumentation Amp	$A_v = (1+2R/R_G) \times R_2/R_1$
INTEGRATORS & DIFFERENTIATORS			
Integrator V_{out}	$V_{out} = -(1/RC) \int V_{in} dt$	Differentiator V_{out}	$V_{out} = -RC \times dV_{in}/dt$
Integrator f_{lower}	$f_{low} = 1/(2\pi R_f C)$ (reset R)	Diff. stability	Add R series with C input
COMPARATORS & OSCILLATORS			

Comparator	$V_{out} = \pm V_{sat}$ (no feedback)	Hysteresis V_H	$V_H = 2V_{sat}R_1 / (R_1 + R_2)$
Schmitt Trigger +	$V^+ = V_{sat}R_1 / (R_1 + R_2)$	Schmitt Trigger -	$V^- = -V_{sat}R_1 / (R_1 + R_2)$
Wien Bridge f_0	$f_0 = 1 / (2\pi RC)$	Phase Shift Osc f_0	$f_0 = 1 / (2\pi\sqrt{6} RC)$
555 Astable f	$f = 1.44 / ((R_1 + 2R_2)C)$	555 Duty Cycle	$D = (R_1 + R_2) / (R_1 + 2R_2)$
ACTIVE FILTERS (see Section 6 for complete filter reference)			
1st-order LPF f_c	$f_c = 1 / (2\pi RC)$	1st-order HPF f_c	$f_c = 1 / (2\pi RC)$
Sallen-Key BPF f_0	$f_0 = 1 / (2\pi\sqrt{R_1R_2C_1C_2})$	MFB Gain	$A_v = -R_2/R_1 @ dc$

OP-AMP CURRENT SOURCE & CURRENT SINK CIRCUITS			
VOLTAGE-CONTROLLED CURRENT SOURCE (Howland Pump)			
Howland Iout	Iout = (Vin2 - Vin1) / R [A] — requires matched resistors R1=R2=R3=R4=R		
Compliance	Max Vload = Vcc - Vsat (op-amp output swing limited)		
Accuracy tip	Use 0.1% matched R; unmatched R creates load-dependent error		
SIMPLE VCCS — GROUNDED LOAD CURRENT SOURCE (NPN + Op-Amp)			
Circuit	Vref—(+)OpAmp—(-)—Base of NPN transistor Emitter—Rset—GND Emitter feedback to (-)OpAmp Load: VCC—Load—Collector		
Output current	Iout = Vref / Rset (op-amp forces V- = V+ = Vref across Rset)		
Example 4-20mA	Iout=4mA: Vref=0.4V,Rset=100Ω Iout=20mA: Vref=2.0V,Rset=100Ω		
Accuracy	Error ≈ Vos/Rset + IB → use low Vos op-amp (OPA2333, LT1013)		
CURRENT SINK — GROUNDED LOAD (Op-Amp + N-MOSFET)			
Circuit	VCC—Load—Drain(NMOS)—Source—Rset—GND Rset mid-point → OpAmp(-) Vctrl → OpAmp(+) OpAmp out → Gate		
Sink current	Isink = Vctrl / Rset (loop forces Vsource = Vctrl via feedback)		
MOSFET choice	Use logic-level N-MOSFET (Vgs(th) < 2.5V) for 3.3V/5V control		
Max Isink	Limited by MOSFET Id(max) and Rset power: P = Isink² × Rset		
4-20mA TRANSMITTER (2-WIRE LOOP-POWERED)			
Loop equation	Iloop = Vsensor_scaled / Rset range 4mA (0%) to 20mA (100%)		
Loop voltage	Vloop = 24V (typical) Vdrop_wire + Vdrop_receiver + VCCS compliance		
Receiver	Vout = Iloop × Rload (Rload = 250Ω → 1-5V output; 500Ω → 2-10V)		
Power budget	Device must operate on 4mA × (24V - losses) ≈ 80mW max		
PRECISION CURRENT REFERENCE (Op-Amp + Zener + BJT)			
Zener reference	Vzener → non-inv input → op-amp → BJT base	Output Iref	Iref = (Vzener - VBE) / Rset
Temperature drift	Match Zener TC with VBE TC for stability	Better option	Use bandgap reference IC (LM4040, REF02)
Current mirror	IC1 = IC2 when VBE1 = VBE2 (matched BJTs)	Wilson mirror	Higher output impedance (3rd BJT added)
CURRENT MEASUREMENT WITH OP-AMP (SHUNT AMPLIFIER)			
High-side sense	Vdiff = I × Rshunt Use diff-amp: Vout = (R2/R1) × I × Rshunt		
Low-side sense	Shunt between load and GND Single-ended amp: Vout = I × Rshunt × Av		
INA219 / INA226	Integrated shunt + 12/16-bit ADC + I2C — for 0-3.2A / ±40V		
NOTE: For 4-20mA industrial loops: use dedicated ICs (XTR115, AD693). Always protect op-amp inputs with back-to-back Schottky clamps on industrial wiring.			

6. FILTERS (Passive & Active)

PASSIVE FILTERS			
RC LPF -3dB	$f_c = 1 / (2\pi RC)$	RC HPF -3dB	$f_c = 1 / (2\pi RC)$
RC LPF $H(j\omega)$	$ H = 1 / \sqrt{1 + (f/f_c)^2}$	RC HPF $H(j\omega)$	$ H = (f/f_c) / \sqrt{1 + (f/f_c)^2}$
LC LPF f_c	$f_c = 1 / (2\pi \sqrt{LC})$	LC BPF f_0	$f_0 = 1 / (2\pi \sqrt{LC})$
RLC BW	$BW = R/L = f_0/Q$	Q factor	$Q = f_0/BW = (1/R) \sqrt{L/C}$
Notch (Twin-T) f_0	$f_0 = 1 / (2\pi RC)$	Notch depth (ideal)	Infinite (infinite Q)
ACTIVE FILTER TYPES			
Butterworth	Maximally flat passband, -20n dB/dec rolloff	Chebyshev	Faster rolloff, passband ripple
Bessel	Maximally flat group delay (best phase linearity)	Elliptic (Cauer)	Steepest rolloff, both ripples
SALLEN-KEY (2nd ORDER)			
S-K LPF f_0	$f_0 = 1 / (2\pi \sqrt{R_1 R_2 C_1 C_2})$	S-K LPF Q	$Q = \sqrt{R_1 R_2 C_1 C_2} / (C_2 (R_1 + R_2))$
S-K HPF f_0	$f_0 = 1 / (2\pi \sqrt{R_1 R_2 C_1 C_2})$	S-K HPF Q	$Q = \sqrt{R_1 R_2 C_1 C_2} / (R_1 (C_1 + C_2))$
Unity-gain S-K	$A_v = 1$ (Butterworth: $Q=0.707$)	Gain >1 S-K	$A_v = 1 + R_b/R_a$
BODE PLOT RULES			
Single pole slope	-20 dB/decade below f_c	Single zero slope	+20 dB/decade above f_z
2nd-order rolloff	-40 dB/decade	Phase at f_c (1P)	-45°
Phase total 1P	0° to -90°	Phase total 2P	0° to -180°
GROUP DELAY & PHASE			
Group Delay τ_g	$\tau_g = -d\phi/d\omega$ [s]	Phase Linearity	Best: Bessel, Worst: Elliptic
Decibels	$dB = 20 \log V_{out}/V_{in} $	Half-power point	-3.01 dB = $ H = 0.707$

7. DIGITAL ELECTRONICS

BOOLEAN ALGEBRA & GATES			
AND	$Y = A \cdot B$	OR	$Y = A + B$
NOT	$Y = A'$	NAND (Universal)	$Y = (A \cdot B)'$
NOR (Universal)	$Y = (A+B)'$	XOR	$Y = A \oplus B = A'B + AB'$
XNOR	$Y = (A \oplus B)' = AB + A'B'$	De Morgan 1	$(AB)' = A' + B'$
De Morgan 2	$(A+B)' = A' \cdot B'$	Consensus	$AB + A'C + BC = AB + A'C$
COMBINATIONAL CIRCUITS			
Half Adder S	$S = A \oplus B$	Half Adder Cout	$Cout = AB$
Full Adder S	$S = A \oplus B \oplus Cin$	Full Adder Cout	$Cout = AB + Cin(A+B)$
MUX 4:1 out	$Y = I0s'1s'0 + I1s'1s0 + I2s1s'0 + I3s1s0$	DEMUX 1:4	Reverse of MUX
Encoder	2^n inputs $\rightarrow$ n outputs	Decoder	n inputs $\rightarrow$ 2^n outputs
Comparator 1-bit	$A=B$: XNOR; $A>B$: AB' ; $A<B$: $A'B$	Priority Encoder	Highest input wins
FLIP-FLOPS & REGISTERS			
D Flip-Flop	$Q_{next} = D$ (edge-triggered)	JK FF toggle	$J=K=1$: toggle; $J=K=0$: hold
SR FF Set	$S=1, R=0$: $Q=1$	SR FF Reset	$S=0, R=1$: $Q=0$
T Flip-Flop	Q toggles when $T=1$ at clock edge	Metastability	Occurs when setup/hold violated
Setup time tsu	Data must be stable before clk edge	Hold time th	Data stable after clk edge
Max clock freq	$f_{max} = 1/(tpd + tsu + tskew)$	Propagation delay	tpd = output delay from clk edge
COUNTERS & STATE MACHINES			
Mod-N counter	Divides by N , needs $\log_2 N$ FFs	Ring counter	N FFs for Mod- N
Johnson counter	$2N$ states with N FFs	Gray code counter	1-bit change per step
Moore FSM	Output = $f(\text{state only})$	Mealy FSM	Output = $f(\text{state}, \text{input})$
NUMBER SYSTEMS & ARITHMETIC			
Binary to Decimal	$\Sigma \text{ bit} \times 2^{\text{position}}$	2's Complement	Invert all bits, add 1
Hex digit	= 4 binary bits (nibble)	BCD	4 bits per decimal digit (0-9)
Signed range n-bit	$-(2^{(n-1)})$ to $2^{(n-1)}-1$	Unsigned range	0 to $2^n - 1$
LOGIC FAMILIES			
TTL VOH	$> 2.4 \text{ V}$	TTL VOL	$< 0.4 \text{ V}$
CMOS logic swing	Rail-to-rail (0 to VDD)	CMOS power	Dynamic only: $P = C f V^2$
Fan-out TTL	10 loads max (std TTL)	Noise Margin	$NMH = V_{OH} - V_{IH}$

8. ANALOGUE ELECTRONICS

SIGNAL ANALYSIS			
RMS General	$V_{rms} = \sqrt{(1/T \int v^2 dt)}$	Average (full-wave)	$V_{avg} = 2V_m/\pi$
Form Factor	$FF = V_{rms}/V_{avg} = 1.11$ (sine)	Crest Factor	$CF = V_{peak}/V_{rms} = \sqrt{2}$ (sine)
THD	$THD = \sqrt{(V_2^2 + V_3^2 + \dots)}/V_1 \times 100\%$	SINAD	Signal+Noise+Distortion ratio
SNR	$SNR = 20\log(V_{signal}/V_{noise})$ dB	Dynamic Range	$DR = 20\log(V_{max}/V_{min})$
FEEDBACK AMPLIFIERS			
Closed-loop gain	$A_{cl} = A_{ol} / (1 + A_{ol}\beta)$	Loop gain T	$T = A_{ol} \times \beta$
Bandwidth increase	$BW_{cl} = BW_{ol} \times (1 + T)$	Distortion reduce	$D_{cl} = D_{ol} / (1 + T)$
Series-series FB	Increases R_{in} , increases R_{out}	Shunt-shunt FB	Decreases R_{in} , decreases R_{out}
Stability criterion	Gain Margin > 6 dB, Phase Margin > 45°	Phase Margin	$\phi_m = 180^\circ - \phi @ \text{unity gain} $
SIGNAL GENERATORS			
Barkhausen Criterion	$ A\beta = 1$ and $\angle A\beta = 0^\circ$	Colpitts f_0	$f_0 = 1/(2\pi\sqrt{LC_{eq}})$
Hartley f_0	$f_0 = 1/(2\pi\sqrt{L_T \times C})$	Crystal Oscillator	$Q = 10,000\text{--}100,000$ (stable)
NOISE IN CIRCUITS			
Thermal Noise V_n	$V_n = \sqrt{(4kTR\Delta f)}$ [V/ $\sqrt{\text{Hz}}$]	Shot Noise I_n	$I_n = \sqrt{(2qI_{DC}\Delta f)}$
1/f Noise	Dominates at low frequency	Noise Figure NF	$NF = 10\log(F_{out}/F_{in})$ dB
Friis Formula	$F_{total} = F_1 + (F_2 - 1)/G_1 + (F_3 - 1)/(G_1G_2)$	Noise Temp	$T_e = T_0(F - 1)$
POWER AMPLIFIERS			
Class A η_{max}	$\eta = 25\%$ (resistive), 50% (transformer)	Class B η_{max}	$\eta = 78.5\%$ ($\pi/4 \times 100\%$)
Class AB	Small bias, crossover distortion fix	Class C	$\eta > 90\%$, used in RF/tuned
Class D (switching)	$\eta > 90\%$, PWM + LC filter output	Class E/F	Resonant switching, RF PAs
Thermal resistance	$T_j = T_a + P_d \times (\theta_{jc} + \theta_{cs} + \theta_{sa})$	Heat sink calc	$\theta_{sa} = (T_j - T_a)/P_d - \theta_{jc} - \theta_{cs}$

ANALOGUE CIRCUIT DESIGN — EXTENDED REFERENCE			
SIGNAL CONDITIONING CHAIN			
Typical chain	Sensor → Anti-alias LPF → Instrumentation Amp → PGA → ADC → DSP/MCU		
Input protection	TVS diode + series R clamp to supply rails	ESD rating	IEC 61000-4-2: $\pm 2\text{kV}$ contact, $\pm 4\text{kV}$ air
Anti-alias LPF	$f_c < f_s/2$ (Nyquist); set $f_c = 0.4 \times f_s$ for margin	Order choice	2nd order Bessel for min phase distortion
Inst. Amp gain	$A_v = 1 + 2R/R_g$ (INA128: $R_g = 50\text{k}\Omega/A_v$)	CMRR typical	80–120 dB at 60Hz
PGA (prog gain amp)	Digitally set gain via SPI/I2C — ADS8688, PGA112	Gain ranging	Auto-range: start high gain, reduce on clip
IMPEDANCE MATCHING			
Max power transfer	$Z_{Source} = Z_{Load}^*$ (conjugate match)	Voltage match	$Z_{Source} \ll Z_{Load}$ (buffer between stages)
Input buffer	Voltage follower: $Z_{in} = \infty$, $Z_{out} \approx 1/\text{gm}$	Output buffer	Emitter follower: $Z_{out} \approx 25\Omega$
Resistive divider load	$V_{out} = V_{in} \times R_2/(R_1 + R_2)$ only valid if $Z_{load} \gg R_2$	Loading error	$\text{Error}\% = R_2 / (Z_{load})$ vs R_2
PRECISION RECTIFIER & PEAK DETECTOR			

Precision HWR	D in feedback: $V_{out} = V_{in}$ for $V_{in} > 0$, $V_{out} = 0$ for $V_{in} < 0$ (no 0.7V drop)		
Full-wave prec.	Two-stage: HWR + summer $V_{out} = V_{in} $ for all V_{in}		
Peak detector	D + C: $V_{cap} = V_{peak}$ Decay: V_{cap} decays at $\tau = R_{leak} \times C$		
RMS converter	AD736/AD737: true RMS IC $V_{out} = V_{rms}$ for any waveform		
SAMPLE & HOLD (S/H)			
Acquisition time	Time for output to settle to $\pm 0.5\text{LSB}$ after sample cmd	Droop rate	$\Delta V / \Delta t = I_{leak} / C_{hold}$
Aperture jitter	Uncertainty in sample instant $\rightarrow$ amplitude error	Clock jitter	$\sigma V = \sigma t \times dV/dt _{max}$
S/H formula	$V_{hold} = V_{in}(t_{sample})$ (frozen for ADC conversion)	Hold cap	$C = I_{ADC} / (dV/dt_{max})$
VOLTAGE REFERENCES			
Bandgap reference	1.25V (LM385) or 2.5V (REF02) $\pm 0.1\%$	Temp coeff	Bandgap: 5-50 ppm/ $^{\circ}\text{C}$; Zener: 50-200 ppm/ $^{\circ}\text{C}$
Buried Zener	Sub-surface Zener: < 5 ppm/ $^{\circ}\text{C}$	Low noise ref	LT6655, ADR4520: $< 1\mu\text{V}$ p-p noise
Shunt vs series ref	Shunt: 2-terminal, constant V (TL431)	Series ref	3-terminal, lower dropout (LM4040)
ANALOGUE MULTIPLICATION & MODULATION			
Analogue multiplier	$V_{out} = (V_1 \times V_2) / V_{ref}$ (AD633, MPY634)	Applications	Power metering, AM modulation, AGC
AM modulation	$V_{out} = V_c(1 + m \times V_m)\sin\omega_c t$ $m = \text{mod. depth}$	DSB-SC	Balanced modulator – suppressed carrier
Lock-in amplifier	Extracts signal at known freq from noisy background	PSRR use	Multiply $\times$ ref, LPF $\rightarrow$ rejects all other f
GROUNDING & SHIELDING BEST PRACTICES			
Star ground	All grounds meet at single point (low frequency)	Ground plane	Solid copper plane (RF / switching circuits)
Guard ring	Driven shield at same potential as hi-Z node	Eliminates	Leakage current across PCB surface
Twisted pair	Differential signal – cancels common-mode noise	Shield drain	Connect shield at one end only (to signal GND)
Decoupling caps	100nF ceramic + 10 μF electrolytic per power pin	Placement	$< 2\text{mm}$ from IC VCC pin; via direct to GND plane
NOTE: Analogue design rule: keep signal traces short, away from switching nodes, and always decouple power locally. A poor ground plane ruins even the best circuit design.			

9. EMBEDDED SYSTEMS

TIMING & CLOCK			
Instruction Time	$T_{inst} = CPI / f_{clk}$	MIPS	$MIPS = f_{clk} / (CPI \times 10^6)$
Timer Period	$T = (PRE \times (TOP+1)) / f_{clk}$	PWM Duty Cycle	$D\% = (OCR / TOP) \times 100$
PWM freq	$f_{PWM} = f_{clk} / (PRE \times (TOP+1))$	Baud rate error%	$err = (actual - desired) / desired \times 100$
ADC & DAC			
ADC Resolution	$LSB = V_{ref} / 2^n$	ADC Output code	$D = (V_{in} \times 2^n) / V_{ref}$
ADC Accuracy	$\pm 0.5 \text{ LSB ideal}$	SNR (n-bit ADC)	$SNR = 6.02n + 1.76 \text{ dB}$
DAC Output V	$V_{out} = (D / 2^n) \times V_{ref}$	Settling time	Time for V_{out} to reach $\pm 0.5 \text{ LSB}$
Nyquist Rate	$f_s \geq 2 \times f_{max} \text{ (signal)}$	Anti-alias filter	$f_c < f_s / 2$
UART / SERIAL			
UART frame	Start(1) + Data(8) + Parity(0/1) + Stop(1-2)	Baud = bit rate	Bits per second
SPI speed	Up to $f_{clk}/2$ (master clock/2)	SPI modes	CPOL/CPHA: 0,0 0,1 1,0 1,1
I2C speed	100kbps / 400kbps / 1Mbps / 3.4Mbps	I2C address	7-bit (128 nodes) or 10-bit
MEMORY MAP & ADDRESSING			
Address decode	$n \text{ address bits} \rightarrow 2^n \text{ locations}$	Word width	Data bus width (8/16/32 bits)
Memory size	$Size = 2^{(address_bits)} \times word_width / 8$	Base+Offset	Effective addr = base + offset
RTOS CONCEPTS			
Task priority	Higher number = higher priority (common)	Context switch	Save/restore CPU state (registers)
Semaphore	Counting sync primitive (binary=mux)	Deadlock	2+ tasks wait on each other's resource
Jitter	Variation in task execution timing	Latency	Time from event to response
Stack overflow	Task stack exceeds allocated RAM	Watchdog timer	WDT resets if not kicked in time
POWER MANAGEMENT			
Dynamic power	$P = CL \times VDD^2 \times f \times \alpha$	Static power	$P = VDD \times I_{leak}$
Sleep modes	Idle→ADC Noise Red.→Power-save→Standby→Power-down	Wakeup sources	INT, WDT, Timer, USART, TWI
NOTE: Always enable watchdog in production firmware. Use volatile for shared vars between ISR and main loop.			

10. MICROCONTROLLERS (MCU)

MCU ARCHITECTURE & PERIPHERALS			
AVR (ATmega)	8-bit Harvard, 1 MIPS/MHz, 2-16KB SRAM	ARM Cortex-M0+	32-bit, 1.77 DMIPS/MHz, low power
ARM Cortex-M4	32-bit+FPU, DSP, 3.4 DMIPS/MHz	ESP32	Dual-core Xtensa 240MHz, WiFi+BT
STM32 families	F0/G0=M0, F1/F3=M3, F4=M4, H7=M7	PIC16/18	8-bit RISC, simple Harvard
TIMERS			
Timer overflow IRQ	Every (2^n - start_val) counts	Input Capture	Records timer count on edge
Output Compare	Action on match (PWM, GPIO toggle)	Encoder mode	Quadrature pulse counting
GPIO & INTERRUPTS			
GPIO output current	Typ 8-40 mA (check datasheet)	GPIO input levels	VIH ≥ 0.7VDD, VIL ≤ 0.3VDD
External INT	EXTI on rising/falling/both edge	NVIC priority	0=highest, check MCU bits (4-8bit)
ISR golden rules	Keep short, no blocking, volatile vars	Debounce	SW delay or RC+Schmitt HW
DMA (Direct Memory Access)			
DMA purpose	Data transfer without CPU (mem↔periph)	Circular mode	Buffer wraps, continuous transfer
Half-transfer IRQ	Process first half while second fills	DMA priority	VHigh > High > Med > Low
BOOTLOADER & PROGRAMMING			
ISP (In-System Prog)	Program via SPI/UART without removal	JTAG/SWD	Debug + program interface
Bootloader address	Start of flash (0x00000 or 0x08000000)	Application address	After bootloader region
Flash endurance	10k-100k write cycles typical	EEPROM endurance	100k-1M write cycles
NOTE: Use DMA for UART/SPI/ADC to avoid CPU stalls. ARM SWD only needs 2 pins vs JTAG 4 pins.			

MICROCONTROLLER — EXTENDED REFERENCE			
POPULAR MCU FAMILIES COMPARISON			
ATmega4809 (AVR)	8-bit, 20MHz, 48KB Flash, 6KB SRAM, UPDI debug	ATtiny3217	8-bit, 20MHz, 32KB Flash, 2KB SRAM
PIC16F877A	8-bit, 20MHz, 14KB Flash, 368B SRAM, ICSP	PIC18F4550	8-bit, 48MHz, 32KB Flash, USB FS OTG
STM32F103C8 (Blue Pill)	32-bit M3, 72MHz, 64KB Flash, 20KB SRAM, 37 GPIO	STM32G0B1	32-bit M0+, 64MHz, 512KB Flash, FDCAN
STM32H755ZI	Dual M7+M4, 480/240MHz, 2MB Flash, 1MB SRAM	RP2040	Dual M0+, 133MHz, 264KB SRAM, PIO blocks
ESP32-S3	Dual Xtensa 240MHz, WiFi+BT5, AI accelerator	nRF52840	M4F, 64MHz, BLE5, USB, 1MB Flash, 256KB SRAM
DSPIC33EP	16-bit DSC, 70 MIPS, hardware QEI, PWM, CAN	TMS320F28379D	32-bit C28x DSP, 200MHz, dual-core, CLA coprocessor
CLOCK CONFIGURATION			
PLL multiply	Fsys = Fxtal × PLL_M / PLL_N / PLL_P	Flash wait states	Add wait states when Fsys > threshold voltage
Clock security CSS	Detects HSE failure, falls back to HSI	HSI accuracy	±1% trimmed factory (STM32) vs crystal ±20ppm

Clock gating	Disable unused peripheral clocks to save power	RCC enable	<code>RCC->APB1ENR = RCC_APB1ENR_TIM2EN;</code>
ADC — ADVANCED CONFIGURATION			
ADC sampling time	$T_{\text{samp}} \geq (R_{\text{in}} + R_{\text{adc}}) \times C_{\text{adc}} \times \ln(2^{(n+1)})$	Min sample time	Drives C_{adc} to required accuracy
Differential mode	Measures $V_+ - V_-$ (rejects common mode noise)	Pseudo-diff	V_- tied to stable ref, V_+ is signal
Oversampling	Average N samples $\rightarrow +\log_2(N)/2$ extra bits	16× oversample	12-bit ADC $\rightarrow$ effective 14-bit resolution
ADC triggering	Timer CC event starts ADC — precise periodic sampling	DMA + ADC	Continuous DMA fill $\rightarrow$ no CPU needed for samples
PWM ADVANCED FEATURES			
Complementary PWM	High + low side with dead time insertion	Dead time calc	$DT = DT_{\text{count}} / F_{\text{clk}}$ (e.g. 100ns = 10 counts @ 100MHz)
Centre-aligned PWM	Counter counts up then down — lower harmonics	Phase shift PWM	Multiple timers synced at phase offsets
SPWM (sine PWM)	Sine modulation: compare sin table vs triangle carrier	THD SPWM	< 5% THD with LC output filter
COMMUNICATION PERIPHERAL DETAILS			
UART flow control	RTS/CTS hardware flow control for high-speed data	UART FIFO	Hardware FIFO prevents overrun (16-64 bytes)
SPI DMA transfer	Set up DMA for TX+RX simultaneously	SPI CS control	Manual CS (GPIO) or hardware NSS managed
I2C clock stretching	Slave holds SCL low to stall master	I2C timeout	Set TIMEOUT register to detect bus lockup
CAN init sequence	Set bit timing, enable, wait INAK=0, set normal mode	CAN bus off	TEC > 255 $\rightarrow$ auto bus-off, requires reset
DEBUGGING & DIAGNOSTICS			
SWD interface	SWDIO + SWDCLK + GND (+Vref optional)	JTAG	TDI+TDO+TMS+TCK+TRST — 5 pins, supports boundary scan
ITM / SWO printf	printf via SWO pin — no UART needed, zero overhead	Live watch	OpenOCD + GDB: watch variables in real-time
Fault handlers	HardFault, MemManage, BusFault, UsageFault — decode CFSR	CFSR register	Precise error: divide by 0, unaligned, invalid PC
Stack watermark	Fill stack with 0xDEADBEEF, check how far it depleted	FreeRTOS	<code>uxTaskGetStackHighWaterMark()</code> returns min free words
PRODUCTION & RELIABILITY			
Watchdog types	IWDG (independent, LSI clock) WWDG (windowed, APB clock)	IWDG timeout	<code>IWDG_PR + IWDG_RLR</code> : 1ms-32s range
Brown-out reset BOR	Resets if VDD falls below threshold	BOR levels	STM32: BOR0-3 (1.8-2.9V configurable)
Option bytes	Configure BOR, WDG, RDP, nBOOT0 in Flash option area	RDP levels	0=open, 1=no debug read, 2=permanent lock
CRC hardware	Built-in CRC32 unit — verify firmware images	Reset cause	<code>RCC->CSR</code> : PINRSTF, BORRSTF, IWDGRSTF, WWDGRSTF
NOTE: Enable BOR, IWDG, and stack watermark monitoring in all production firmware. Use ITM/SWO during development — remove for release. Always decode HardFault CFSR in the fault handler.			

11. SEMICONDUCTOR MEMORIES

MEMORY TYPES COMPARISON			
SRAM	Static, 6T cell, fast (ns), volatile, costly	DRAM	Dynamic, 1T+1C, needs refresh, dense
SDRAM	Synchronous DRAM, burst access, DDR/DDR4	EEPROM	Electrically erasable, byte-write, slow
NOR Flash	Random access, fast read, slow write/erase	NAND Flash	Page/block access, dense, fast write
FRAM (FeRAM)	Non-volatile, 10^{12} writes, fast	MRAM	Magnetic, non-volatile, unlimited writes
KEY PARAMETERS			
Access time t_a	Address valid → data valid	Cycle time t_c	Min time between accesses ($t_c \geq t_a$)
Read latency CL	Clock cycles to first data (DRAM)	Burst length BL	Words transferred per command
Endurance	Erase/write cycles before wear-out	Retention	Years data retained without power
Write amplification	Physical writes / logical writes	Wear leveling	Distribute writes across flash cells
MEMORY HIERARCHY			
L1 Cache	On-die, ~1ns, 16-64 KB per core	L2 Cache	On/near die, ~5ns, 256KB-4MB
L3 Cache	Shared, ~15ns, 4-64 MB	Main RAM	Off-chip DRAM, ~60ns, 4-64 GB
Cache hit rate H	$H = \text{hits} / \text{total accesses}$	AMAT	$H \times T_c + (1-H) \times T_m$
Direct-mapped	1-way, simple, conflict misses	N-way set-associative	N lines per set, LRU eviction
NAND FLASH OPERATIONS			
Page size	512B - 16KB typical	Block = N pages	Erase unit (64-256 pages typical)
ECC	Error correction required (BCH/LDPC)	Bad block mgmt	Mark bad at factory + runtime
FTL	Flash Translation Layer: logical-physical	JEDEC standards	eMMC, UFS, SD card interfaces

PARALLEL MEMORY INTERFACES			
PARALLEL BUS SIGNALS			
Address bus $A[n:0]$	Selects memory location: 2^n total locations	Data bus $D[m:0]$	Width $m+1$ bits (8/16/32 bit wide)
Chip Enable /CE	Activates device (active low)	Output Enable /OE	Enables data output onto bus (active low)
Write Enable /WE	Enables write cycle (active low)	Byte Enable /BEx	Selects byte lane on 16/32-bit wide bus
PARALLEL SRAM (e.g. IS62WV25616BLL — 256Kx16)			
Access time t_{AA}	Address stable → valid data: 45-70ns typical	CE access t_{ACS}	CE low → valid data: same as t_{AA}
Cycle time t_{RC}	Min time between read starts ($\geq t_{AA}$)	Write cycle t_{WC}	Min write cycle time
Read cycle	/CE low + /OE low → data valid after t_{AA}	Write cycle	/CE low + /WE pulse → data latched on /WE rise
Density example	IS62WV25616: 256K addr × 16-bit = 512KB	Bus interface	STM32 FMC/FSMC — auto generates timing signals
PARALLEL NOR FLASH (e.g. S29GL064S — 64Mbit)			

Read access	Same as SRAM read — direct random access (XIP)	Sector erase	Cmd sequence: 0xAA→0x55→0x80→0xAA→0x55→0x30
Word program	Cmd sequence: 0xAA→0x55→0xA0→data	Program time	≈ 9μs per word typical
CFI query	Common Flash Interface: ID geometry + timing params	Erase time	≈ 500ms per sector (64KB)
XIP (execute)	CPU fetches instructions directly from NOR (no copy)	Bus width	x8 (byte wide) or x16 (word wide)
PARALLEL DRAM / SDRAM (e.g. W9825G6KH — 256Mb)			
RAS (Row Address Str)	Latches row address on CAS pin bundle	CAS (Col Address)	Latches column address after RAS
Refresh requirement	Each row must be refreshed every 64ms (4096 rows)	Auto-refresh	SDRAM controller sends AUTO REFRESH cmd
CAS latency CL	Clocks from READ cmd to first data: CL=2 or 3	Burst mode	Up to 8 words per command cycle
RAS to CAS delay tRCD	Min time between RAS and CAS assertion	Row precharge tRP	Time to deactivate row before next activate
SDRAM Init seq	Power-on → 200μs wait → PRECHARGE ALL → 8× AUTO-REF → LOAD MODE REG	Mode reg	Set CL, burst length, burst type
STM32 FMC (Flexible Memory Controller) CONFIGURATION			
FMC banks	Bank1: NOR/SRAM Bank2: NAND Bank3: SDRAM	Address mapping	NE1: 0x60000000, NE2: 0x64000000...
SRAM timing regs	ADDSET, ADDHLD, DATAST, BUSTURN in BCR/BTR	SDRAM timing	TMRD, TXSR, TRAS, TRC, TWR, TRP, TRCD in SDTR
FMC clock	SDRAM: FMC_CLK = HCLK/2 or /3 (max 100MHz)	Refresh rate	Set FMC_SDRTR: Count = (tREFI × FMC_CLK) - 20
PARALLEL vs SERIAL MEMORY COMPARISON			
Parallel SRAM	Fast (≤70ns), simple, many pins (26-32 signal lines)	Serial SRAM (SPI)	Slower, 4-6 pins, up to 104MHz SPI
Parallel NOR	XIP capable, fast read, many pins	Serial NOR (QSPI)	4-bit QSPI: execute-in-place via memory-mapped
Use parallel when	< 100ns latency required, large bandwidth (LCD, DSP)	Use serial when	Pin-count constrained, medium speed OK
NOTE: Most modern MCUs use QSPI/OctoSPI for external Flash (fewer pins, DMA, memory-mapped XIP). Use parallel SRAM only when access time < 70ns is critical or when existing PCB dictates it.			

12. INDUSTRIAL COMMUNICATION PROTOCOLS

MODBUS			
Modbus RTU	Serial (RS-232/485), binary, CRC-16	Modbus TCP	Ethernet, port 502, no CRC (TCP handles)
Modbus ASCII	Serial, ASCII chars, LRC checksum	Max slaves RTU	247 addresses (1-247)
Function 01	Read Coils (discrete output)	Function 02	Read Discrete Inputs
Function 03	Read Holding Registers (16-bit)	Function 04	Read Input Registers
Function 05	Write Single Coil	Function 06	Write Single Register
Function 15	Write Multiple Coils	Function 16	Write Multiple Registers
RTU timeout	3.5 char silence = frame boundary	Modbus frame	Addr(1)+FC(1)+Data(N)+CRC(2)
CAN BUS			
CAN 2.0A (std)	11-bit identifier, up to 1 Mbps	CAN 2.0B (ext)	29-bit identifier
CAN FD	Up to 8 Mbps data phase, 64-byte payload	Dominant bit	Logic 0 (bus driven low)
Arbitration	CSMA/CA: lowest ID wins (non-destructive)	ACK bit	Receiver pulls dominant
Max nodes	~127 (practical, bus load dependent)	Max cable (1Mbps)	~40 m
CRC	CAN: 15-bit CRC, FD: 17/21-bit	Termination	120Ω at each end of bus
PROFIBUS & PROFINET			
PROFIBUS DP	RS-485, up to 12 Mbps, 126 nodes	PROFIBUS PA	MBP, intrinsically safe, 31.25 kbps
PROFINET	Industrial Ethernet (100M/1G), IRT/RT/NRT	IRT cycle time	< 1 ms (sync'd Ethernet)
GSD file	Device description for config tool	Slot/Module	PROFIBUS modular addressing
OTHER FIELD BUS PROTOCOLS			
DeviceNet	CAN-based, 500kbps, 64 nodes, ODVA	EtherNet/IP	CIP over Ethernet, ODVA
EtherCAT	Ethernet slave-to-slave, <100μs cycle	Modbus RTU baud	9.6k / 19.2k / 38.4k / 115.2k
RS-232 levels	±3 to ±15V, point-to-point, <15m	RS-485 levels	±1.5V diff, 32-256 nodes, 1200m
RS-422	Differential, full-duplex, 1200m @ 100kbps	LIN bus	Single-wire, 20kbps, automotive
NOTE: Modbus is the most widely used in Africa/SA industry. CAN is dominant in automotive. PROFIBUS/PROFINET in Siemens PLC environments.			

13. DC METERING

DC MEASUREMENT BASICS			
Voltmeter Rin	$R_{in} \rightarrow \infty$ ideal (typ >10M Ω)	Ammeter Rin	$R_{in} \rightarrow 0$ ideal (typ <1 Ω)
Loading error %	$err = R_{in_meter} / (R_{in_meter} + R_{circuit}) - 1$	4-wire (Kelvin)	Eliminates lead resistance in R meas.
SHUNTS & MULTIPLIERS			
Ammeter shunt Rs	$R_s = I_{fs} \times R_m / (I - I_{fs})$	Voltmeter Rm_series	$R_{series} = (V/I_{fs}) - R_m$
Shunt power	$P_s = I^2 \times R_s$	Multiplier factor	$m = I_{full_scale} / I_{fs}$
BRIDGE CIRCUITS			
Wheatstone balance	$R1/R2 = R3/Rx \rightarrow Rx = R2R3/R1$	Sensitivity S	$S = \Delta V_{out} / \Delta R/R$
Kelvin bridge	Low resistance measurement (<1 Ω)	Maxwell bridge	Measures inductance L
Schering bridge	Measures capacitance & tan δ	Wien bridge	Frequency measurement
CURRENT SENSING			
Shunt resistor	$V = I \times R_{shunt}$ (typ 1-100 m Ω)	Hall-effect sensor	Isolated, $I = B \times d / (\mu_0 \times Hall\ const)$
CT (Current Transf)	$I_s = I_p \times N_p / N_s$ (ideal)	Rogowski coil	di/dt measurement, no core sat.
DC POWER METERS			
DC Power	$P = V \times I$	Energy meter count	$E = P \times t$ [Wh or kWh]
Battery SOC	Coulomb counting: $SOC = SOC_0 + \int I\ dt / Q_{rated}$	Battery SoH	$SoH = Q_{actual} / Q_{nominal} \times 100\%$
Resistance (4W)	$R = V_{sense} / I_{force}$	DMM accuracy	% reading + % full scale + digits
NOTE: Always use 4-wire Kelvin connection for $R < 1\Omega$. CT secondary must NEVER be open-circuited (dangerous HV).			

14. AC METERING

AC MEASUREMENT FUNDAMENTALS			
True RMS V	$V_{rms} = \sqrt{(1/T) \int v^2 dt}$	True RMS I	$I_{rms} = \sqrt{(1/T) \int i^2 dt}$
True Power P	$P = (1/T) \int v \times i\ dt$ [W]	Reactive Power Q	$Q = \sqrt{(S^2 - P^2)}$ [VAR]
Apparent Power S	$S = V_{rms} \times I_{rms}$ [VA]	Power Factor PF	$PF = P/S = \cos\phi$
Displacement PF	$DPF = \cos(\phi_1)$ (fundamental only)	True PF (non-linear)	$PF = DPF / \sqrt{(1+THD^2)}$
ENERGY METERING (IEC 62053 / SANS 474)			
Active energy	$Wh = P \times t$ (kWh for billing)	Reactive energy	$VARh = Q \times t$
Class 0.5S	$\pm 0.5\%$ accuracy (utility meters)	Class 1	$\pm 1\%$ (industrial/commercial)
Meter constant	Pulses per kWh (e.g. 1600 imp/kWh)	Calibration pulse	LED/optical output for testing
SA Utility std	NRS 057 / NRS 097 smart metering	Tamper detect	Magnetic, neutral switch, reversal
INSTRUMENT TRANSFORMERS			
CT ratio	e.g. 100/5A (turns ratio = 20)	CT class 0.5	$\pm 0.5\%$ ratio error (metering CT)
VT/PT ratio	e.g. 11kV/110V (step-down)	VT burden	Rated VA load on secondary

CT burden	Max impedance on secondary (Ω or VA)	CT saturation	Avoid: use C-class protection CTs
POWER QUALITY PARAMETERS			
THD-V limit	< 5% total (IEC 61000-2-2)	Flicker Pst/Plt	Short/long-term flicker severity
Voltage unbalance	$VUF\% = V_{neg}/V_{pos} \times 100\%$	Frequency tolerance	50 Hz $\pm$ 0.5 Hz (NRS 048)
Harmonics	$f_n = n \times f_0$ (n=1 fund; 3,5,7 odd dominant)	Notch depth	Line notch depth < 20% (IEEE 519)
Sag (dip)	RMS drop 10-90%, 10ms-1min	Swell	RMS rise > 110%, < 1min
NOTE: South Africa: NRS 048-2 defines PQ limits. Eskom supply frequency 50Hz. Metering to NRS 057 / SANS 62053 standards.			

15. WEBSOCKETS

WEBSOCKET PROTOCOL (RFC 6455)			
Protocol	ws:// (plain) wss:// (TLS)	Default ports	ws: 80, wss: 443
Upgrade header	HTTP GET + Upgrade: websocket	Sec-WS-Key	Base64(16 random bytes)
Sec-WS-Accept	Base64(SHA1(key + GUID))	GUID	258EAF5-E914-47DA-95CA-C5AB0DC85B11
101 response	HTTP 101 Switching Protocols	Full-duplex	Simultaneous send & receive
FRAME FORMAT			
Frame header	FIN(1)+RSV(3)+Opcode(4)+MASK(1)+Len(7)	Payload len	7-bit; 126→16-bit ext; 127→64-bit ext
Opcode 0x1	Text frame (UTF-8)	Opcode 0x2	Binary frame
Opcode 0x8	Close frame	Opcode 0x9/0xA	Ping / Pong (keepalive)
Client masking	Client→Server always masked (XOR key)	Server masking	Server→Client unmasked
CONNECTION LIFECYCLE			
Handshake	HTTP→WS upgrade (one-time)	Heartbeat	Ping/Pong every 30-60s typical
Close handshake	Close frame exchange before TCP close	Status 1000	Normal closure
Status 1001	Going away (server restart)	Status 1008	Policy violation
Reconnect strategy	Exponential backoff: 1s,2s,4s,8s,max32s	Sub-protocols	Sec-WebSocket-Protocol header
PERFORMANCE & SCALING			
Overhead vs HTTP	2-14 bytes/frame vs 200-800 bytes HTTP header	Max connections	~65k per server IP:port (OS limit)
Load balancing	Sticky sessions required (same server)	STOMP	Messaging protocol over WebSocket
Socket.IO	WS + long-polling fallback + rooms	Binary WS	ArrayBuffer / Blob for raw data

16. RESTful API

REST PRINCIPLES (Richardson Maturity Model)			
REST Level 0	Single URI, all via POST (SOAP/RPC-like)	REST Level 1	Resource URIs (per-entity)
REST Level 2	HTTP verbs (GET/POST/PUT/DELETE)	REST Level 3	HATEOAS (self-describing links)
Stateless	No client session on server (each req complete)	Cacheable	Responses declare cacheability
Uniform Interface	Standard HTTP methods + media types	Layered System	Client unaware of intermediaries
HTTP METHODS			
GET	Read resource (safe + idempotent)	POST	Create (not idempotent)
PUT	Full update/replace (idempotent)	PATCH	Partial update (not always idempotent)
DELETE	Remove resource (idempotent)	HEAD	GET without body (check existence)

OPTIONS	Returns allowed methods (CORS preflight)	Idempotent	Same result regardless of repeat calls
HTTP STATUS CODES			
200 OK	Success	201 Created	POST success, Location header
204 No Content	DELETE/PUT success, no body	400 Bad Request	Invalid input/syntax
401 Unauthorized	Auth required (missing/invalid token)	403 Forbidden	Auth OK but insufficient rights
404 Not Found	Resource does not exist	409 Conflict	Duplicate / state conflict
422 Unprocessable	Validation failed	429 Too Many Requests	Rate limit exceeded
500 Internal Error	Server fault	503 Service Unavail.	Maintenance / overload
AUTHENTICATION & SECURITY			
JWT structure	Header.Payload.Signature (Base64url)	JWT verify	HMAC-SHA256 or RSA signature
Bearer token	Authorization: Bearer <token>	Refresh token	Long-lived, rotated on use
OAuth 2.0 flow	Auth Code → Token Exchange → Access	API Key	Simple, static, no expiry
HTTPS	All APIs must use TLS 1.2+ (no HTTP)	CORS	Access-Control-Allow-Origin header
API DESIGN BEST PRACTICES			
Versioning	/api/v1/resource or Accept: vnd+v1	Pagination	?page=1&limit=50 or cursor-based
Filtering	?status=active&type=sensor	Sorting	?sort=created_at&order=desc
Rate limiting	X-RateLimit-Limit / Remaining / Reset	Idempotency key	Idempotency-Key: <uuid> (POST)

17. PoE STANDARDS (Power over Ethernet)

IEEE PoE STANDARDS OVERVIEW			
IEEE 802.3af (PoE)	Max 15.4W PSE / 12.95W PD	IEEE 802.3at (PoE+)	Max 30W PSE / 25.5W PD
IEEE 802.3bt Type 3	60W PSE / 51W PD (4-pair)	IEEE 802.3bt Type 4	90W PSE / 71.3W PD
Cisco UPOE	Proprietary 60W (pre-bt)	PoE++ (bt)	Also called Hi-PoE
ELECTRICAL PARAMETERS			
PSE Voltage	44-57 VDC (af/at) / 50-57V (bt)	PD Input Voltage	37-57V (after cable drop)
Cable resistance	~12.5Ω/100m (Cat5e pair)	Max cable drop	7V typical @ rated current
Pair usage (af/at)	Alt A: pins 1,2,3,6 Alt B: 4,5,7,8	bt pair usage	All 4 pairs simultaneously
PD signature R	25 kΩ (±1kΩ) detection R	Classification	0-1mA to 73mA pulses (Class 0-8)
POWER CLASSES			
Class 0 (af)	0.44-12.95W (unclassified)	Class 1 (af)	0.44-3.84W
Class 2 (af)	3.84-6.49W	Class 3 (af/at)	6.49-25.5W
Class 4 (at)	12.95-25.5W	Class 5 (bt)	up to 40W at PD
Class 6 (bt)	up to 51W at PD	Class 7 (bt)	up to 62W at PD
Class 8 (bt)	up to 71.3W at PD	LLDP / CDP negotiation	Software power class override
CONNECTOR TYPES, CURRENT RATINGS & CABLE SPECIFICATIONS			
RJ-45 (8P8C)	Standard connector – all PoE standards (af/at/bt)	Contact rating	1.5 A per pin max (UL/IEC)

Max current (af/at)	600 mA per pair (2 conductors)	Max current (bt)	960 mA per pair – all 4 pairs active
Total bt conductor I	Up to 960 mA per conductor	Total bt port current	~1.92 A aggregate (4 pairs)
Cat5e (24 AWG UTP)	100Ω ±15%, max 100m, up to 30W safe	Max I Cat5e	360 mA/conductor @ 60°C ambient
Cat6 (23 AWG UTP)	100Ω ±15%, lower DCR, up to 60W	Max I Cat6	600 mA/conductor, lower temp rise
Cat6A (23 AWG UTP/STP)	Required for 802.3bt Type 3/4 (90W)	Max I Cat6A	720 mA/conductor, best thermal perf
Cat7 / Cat8 (shielded)	STP/SFTP, Cat8=40GbE/30m; PoE supported	Connector	Still RJ-45 plug on field end
M12 D-coded (4-pin)	IP67 industrial Ethernet, up to 1GbE	Max I M12-D	4 contacts × 4A (power pins shrd)
M12 X-coded (8-pin)	IP67/IP69K, 10GbE + PoE bt, IEC 61076-2-109	Max I X-coded	Up to 4A per power contact
M12 L-coded	Emerging: PoE + data + power combined	Use case	Industrial robots, AGVs, field sensors
HDBaseT (RJ-45)	Proprietary – up to 100W / 100m	Use case	AV-over-IP, displays, collaboration
Cisco UPOE (RJ-45)	CDP/LLDP negotiates 60W on 4 pairs	Use case	High-power IP phones, thin clients
CABLE THERMAL DERATING			
Bundled cable derating	≥24 cables bundled: derate 50% current	Conduit	Per NEC 310.15 / IEC 60364-5-52
Cat5e 100m loss @ 15W	≈1.2W cable dissipation	Cat6A 100m @ 90W	≈8.1W cable loss – verify budget
Temp rise formula	$\Delta T = I^2 \times R_{loop} \times \theta_{thermal}$	R loop (Cat5e 100m)	~5Ω per pair (both conductors)
DESIGN RULES			
PSE midspan	In-line injector (no PoE switch)	PSE endpoint	PoE integrated in managed switch
PD isolation	1500V galvanic isolation required	Inrush limit	PD must limit inrush < 400mA
Power budget rule	$PSE_{total} \times \eta \geq \Sigma PD_{watts}$	Headroom	Always design with 20% spare
NOTE: Use Cat6A for all new 802.3bt installations. In SA harsh environments (mines, outdoors) use M12 X-coded IP67 connectors. Verify per-port AND total switch chassis PoE budget separately.			

SMART PoE — NEGOTIATED VOLTAGE MODES & TYPES			
STANDARD IEEE 802.3 PoE VOLTAGE NEGOTIATION			
Passive PoE	Fixed voltage (12V/24V/48V) – NO detection/classification	Risk	Always-on – can damage non-PoE devices
IEEE 802.3af Active	Detection → Classification → 48V (44-57V) power-on	Safe	Only powers devices that pass detection
IEEE 802.3at Active	Same detection + 2-event Class for 30W	Voltage	44-57V PSE output
IEEE 802.3bt Active	Physical layer + LLDP/CDP negotiation for 60W/90W	Voltage	50-57V PSE output
NON-STANDARD SMART PoE VOLTAGE SYSTEMS			
24V Passive PoE	Ubiquiti airMAX default: 24V on pins 4,5(+) 7,8(-)	Max power	≈24V × 1A = 24W (cable limited)
48V Passive PoE	Cameras, APs: 48V on 4,5(+) 7,8(-) or 1,2(+) 3,6(-)	Compatibility	Only compatible devices – always check first
Ubiquiti 24V PoE	Proprietary 24V passive – incompatible with 802.3af/at	Adapter	Use Ubiquiti POE-24-24W or INS-8023AF-I

Cisco UPOE (60W)	CDP negotiated – extends 802.3at to 60W using all pairs	Voltage	50-57V, same as standard
PoE++(bt) auto-neg.	LLDP-MED + 802.3bt Layer-1 class – up to 90W	Class 8 Vport	51-57V at port, derates with cable
HDBaseT Alliance	Up to 100W / 100m – 5Play (video+audio+USB+Eth+PoE)	Voltage	Proprietary, NOT 802.3 compatible
PoE over Coax	e-PoC (Ethernet over Coax + power) – BNC connector	Voltage	48V DC on coax; used in CCTV retrofit
Industrial 24V DC PoE	PLC-side: 24V DC supply + Ethernet separate (not true PoE)	Standard	IEC 62443 industrial networks – IS1 separation
VOLTAGE AT PD vs CABLE LENGTH (802.3af/at)			
PSE output	44-57V DC (typical 48V nominal)	Cable drop	$\Delta V = I \times R_{\text{loop}}$ ($R_{\text{loop}} \approx 25\Omega/100\text{m Cat5e}$)
PD min at 0m	44V	PD min at 100m	~37V (after max cable drop at rated current)
PD must accept	37-57V input range (bridge rectifier + DCDC converter)	Efficiency	PD converter η typically 85-92%
Power at PD (af)	$12.95\text{W} = 15.4\text{W} \times (1 - \text{cable_loss fraction})$	Power at PD (at)	25.5W delivered after cable loss
SMART POE DETECTION & CLASSIFICATION SEQUENCE			
Step 1 Detect	PSE applies 2.7-10V probe PD must present 25k Ω $\pm$ 1k Ω signature resistor		
Step 2 Class	PSE applies 15.5-20.5V PD draws class current: Class0=0-4mA, Class3=26-30mA, Class8=58-73mA		
Step 3 Power-on	PSE ramps to full voltage (44-57V) after valid class detected		
Step 4 LLDP	Optional: LLDP-MED packets negotiate exact power level (bt Class5-8)		
Step 5 Monitor	PSE monitors load: if I < 10mA for >300ms $\rightarrow$ PSE removes power (device disconnect)		
NOTE: Never mix passive PoE voltages on the same switch — 24V and 48V passive PoE will damage incompatible devices. Always use active IEEE 802.3 PoE where possible. For SA installations: verify that PoE injectors/switches are 230V-rated (50Hz).			

18. MEMORY MAP — ROM, RAM, SRAM, EEPROM

Region / Type	Base Address	End Address	Size	Notes / Contents
STM32H7 (Cortex-M7 + M4) MEMORY MAP — Typical @ 0x00000000 Base				
FLASH (ROM) — Execute-in-Place, non-volatile				
Flash Bank 1	0x0800 0000	0x080F FFFF	1 MB	Main program code / ISR vectors
Flash Bank 2	0x0810 0000	0x081F FFFF	1 MB	Dual-bank (OTA, swap)
Flash OTP	0x1FF0 7800	0x1FF0 7A0F	528 B	One-time programmable area
Flash Option	0x1FF0 7800	0x1FF0 783F	64 B	RDP, BOR, WDG config bits
SRAM — Tightly Coupled & General Purpose (volatile, fast)				
DTCM-RAM	0x2000 0000	0x2001 FFFF	128 KB	M7 data TCM — 0-wait state, DMA-able
ITCM-RAM	0x0000 0000	0x0000 FFFF	64 KB	M7 instr TCM — fastest fetch
AXI SRAM (D1)	0x2400 0000	0x2407 FFFF	512 KB	General purpose, DMA2D, MDMA
SRAM1 (D2)	0x3000 0000	0x3001 FFFF	128 KB	Peripheral DMA bus
SRAM2 (D2)	0x3002 0000	0x3003 FFFF	128 KB	Peripheral DMA bus
SRAM3 (D2)	0x3004 0000	0x3004 FFFF	32 KB	USB & Ethernet DMA
SRAM4 (D3)	0x3800 0000	0x3800 FFFF	64 KB	Low-power domain, BDMA
Backup SRAM	0x3880 0000	0x3880 0FFF	4 KB	Retained in VBAT / Standby
EEPROM (Emulated in Flash) / External EEPROM				
Emulated EE	0x0808 0000	0x0809 FFFF	128 KB	Flash pages used as EEPROM (HAL_FLASH_EE)
Ext I2C EEPROM	Device addr	0x50–0x57	1–512 KB	AT24Cxx: 7-bit I2C, 400 kHz
Ext SPI EEPROM	CS select	0x00–0xFFFF	64 KB	25LCxxx series, SPI, 10M writes
PERIPHERAL REGISTERS (Memory-Mapped I/O)				
APB1 Periph	0x4000 0000	0x4000 7FFF	32 KB	UART2-8, SPI2-3, I2C, TIM2-7
APB2 Periph	0x4001 0000	0x4001 7FFF	32 KB	UART1, SPI1, TIM1/8, ADC1-3
AHB1 Periph	0x4002 0000	0x4002 1FFF	8 KB	DMA1/2, DMAMUX, ETH, MDMA
AHB2 Periph	0x4802 0000	0x4802 1FFF	8 KB	GPIO A-K, RNG, SDMMC
AHB3 Periph	0x5100 0000	0x5100 3FFF	16 KB	MDMA, JPEGC, LTDC, QUADSPI
Cortex-M SCS	0xE000 E000	0xE000 EFFF	4 KB	NVIC, SysTick, SCB, MPU, FPU
GENERIC MCU MEMORY MAP RULES				
AVR ATmega328P Flash	0x0000	0x7FFF	32 KB	Harvard: code space (word addressed)
AVR ATmega328P SRAM	0x0100	0x08FF	2 KB	Data + stack (grows downward)
AVR ATmega328P EEPROM	0x000	0x3FF	1 KB	8-bit EEPROM, 100k write cycles
ESP32 IRAM	0x4007 0000	0x4009 FFFF	192 KB	Instruction RAM (time-critical ISR)
ESP32 DRAM	0x3FFB 0000	0x3FFF FFFF	320 KB	Data RAM (heap+stack)
ESP32 Flash	0x400C 2000	0x40BF FFFF	~4 MB	External SPI Flash, MMU-mapped

18b. BITWISE DATA SCALING — 64-bit Down to 4-bit

BIT-WIDTH TYPES, RANGES & SCALING OPERATIONS			
uint64_t	0 to 18,446,744,073,709,551,615	int64_t	-9.22×10 ¹⁸ to +9.22×10 ¹⁸
uint32_t	0 to 4,294,967,295	int32_t	-2,147,483,648 to 2,147,483,647
uint16_t	0 to 65,535	int16_t	-32,768 to 32,767
uint8_t	0 to 255	int8_t	-128 to 127
uint4_t (nibble)	0 to 15 (4 bits)	Nibble extract	(val >> shift) & 0x0F
SHIFT OPERATIONS — EXTRACT ANY FIELD FROM 64-BIT VALUE			
64→32 upper half	uint32_t hi = (uint32_t)(val >> 32);	64→32 lower half	uint32_t lo = (uint32_t)(val & 0xFFFFFFFF);
64→16 byte[3]	(uint16_t)((val >> 48) & 0xFFFF)	64→16 byte[2]	(uint16_t)((val >> 32) & 0xFFFF)
64→16 byte[1]	(uint16_t)((val >> 16) & 0xFFFF)	64→16 byte[0]	(uint16_t)(val & 0xFFFF)
64→8 byte n	(uint8_t)((val >> (n*8)) & 0xFF)	Loop all 8 bytes	for(i=7;i>=0;i--) b[i]=val>>(i*8)
64→4 nibble n	(uint8_t)((val >> (n*4)) & 0x0F)	Total nibbles	16 nibbles in a 64-bit value
BITWISE OPERATORS — C SYNTAX			
AND &	Mask / clear bits: val & 0x0F	OR	Set bits: val 0x80
XOR ^	Toggle bits: val ^ 0xFF	NOT ~	Invert all: ~val
Left shift <<	val << n ≡ val × 2 ⁿ (fast multiply)	Right shift >>	val >> n ≡ val / 2 ⁿ (unsigned)
Set bit n	val = (1ULL << n)	Clear bit n	val &= ~(1ULL << n)
Toggle bit n	val ^= (1ULL << n)	Test bit n	if (val & (1ULL << n))
SCALING — PROPORTIONAL VALUE MAPPING			
Linear scale	out = (in * out_max) / in_max	ADC→Engineering	V = (adc_raw * Vref) / (2 ^{bits} - 1)
64-bit to 4-bit	nibble = (uint8_t)((val * 15ULL) / in_max) & 0x0F	4-bit to percent	pct = (nibble * 100) / 15
Fixed-point Q16	val_fp = (int32_t)(float_val * 65536.0f)	Q16 back to float	f = (float)val_fp / 65536.0f
Q format Q(m,n)	n frac bits: LSB = 2 ⁻ⁿ ; range ±2 ^(m-1)	Q15 range	-1.0 to +0.99997 (DSP audio)
FULL EXAMPLE — 64-BIT SENSOR WORD DECODE			
64-bit word layout	[63:48] Timestamp [47:32] Temp×100 [31:16] Voltage×1000 [15:8] Status [7:4] Mode [3:0] Channel		
Extract Channel	uint8_t ch = (uint8_t)(sensor_word & 0x0F);		
Extract Mode	uint8_t mode = (uint8_t)((sensor_word >> 4) & 0x0F);		
Extract Status	uint8_t stat = (uint8_t)((sensor_word >> 8) & 0xFF);		
Extract Voltage	float vout = ((sensor_word >> 16) & 0xFFFF) / 1000.0f;		
Extract Temp	float temp = (int16_t)((sensor_word >> 32) & 0xFFFF) / 100.0f;		
Extract Time	uint16_t ts = (uint16_t)((sensor_word >> 48) & 0xFFFF);		
NOTE: Always use 1ULL<<n (not 1<<n) for 64-bit shifts. Use uint64_t for portability. On 32-bit MCUs, 64-bit ops compile to 2 instructions.			

19. ARM CORTEX-M7 DUAL CORE (STM32H7)

CORE ARCHITECTURE COMPARISON			
Core	Cortex-M7 (CM7) — Master	Core	Cortex-M4 (CM4) — Co-processor
Architecture	ARMv7-M, 6-stage superscalar	Architecture	ARMv7-M, 3-stage pipeline
Max Clock	480 MHz (STM32H755)	Max Clock	240 MHz (STM32H755)
Performance	3.4 DMIPS/MHz + 7.67 CoreMark/MHz	Performance	1.25 DMIPS/MHz + 3.34 CoreMark/MHz
FPU	Double precision (DP-FPU) + DSP SIMD	FPU	Single precision (SP-FPU) + DSP
TCM	64KB ITCM + 128KB DTCM (0-wait)	TCM	No TCM
Cache	16KB I-cache + 16KB D-cache	Cache	No cache (direct SRAM)
MPU	16 regions	MPU	8 regions
BOOT & STARTUP SEQUENCE (STM32H755)			
Boot pins BOOT0	0: Flash boot (normal)	BOOT0=1	System memory (ROM bootloader)
CM7 boots first	Starts at 0x08000000 (Flash Bank 1)	CM4 held	In reset until CM7 releases
CM7 releases CM4	HAL_RCCEx_EnableBootCore(RCC_BOOT_C2)	CM4 starts	From 0x08100000 (Bank 2) or RAM
CM7 vector table	VTOR = 0x08000000	CM4 vector table	VTOR = 0x08100000 (or configured)
INTER-CORE COMMUNICATION (AMP — Asymmetric Multi-Processing)			
HSEM (HW Semaphore)	32 semaphores for mutual exclusion	HSEM take	HSEM->RLR[n] == COREID<<8 PROCID
HSEM ISR notify	HSEM_IRQn triggers on release	Shared memory	Use SRAM1/SRAM2/AXI SRAM (no cache)
OpenAMP/virtIO	RPMSG for message-based IPC	Mailbox	IPCC peripheral: 6 channels bidirec.
Cache coherency	CM7 must flush/inv D-cache before sharing	D-cache flush	SCB_CleanDCache_by_Addr(ptr, size)
D-cache invalidate	SCB_InvalidateDCache_by_Addr(ptr, sz)	CM4 rule	CM4 accesses SRAM4/D3 directly (no cache issue)
CLOCK DOMAINS			
D1 domain (CM7)	480 MHz max, AXI/AHB3, LTDC, GPU	D2 domain (CM4)	240 MHz, APB1/APB2, UART, SPI, I2C
D3 domain	Low-power, runs when D1/D2 in Standby	HSE crystal	25 MHz external → PLL → sysclk
PLL1	→ CM7 core + AXI + Flash	PLL2	→ USB, ADC, SDMMC, SPI
PLL3	→ UART, I2C, LP timers	LSE	32.768 kHz → RTC, LPTIM
MEMORY PROTECTION UNIT (MPU) ATTRIBUTES			
TEX+C+B+S	Cache policy bits for each region	XN (eXec Never)	Prevent code exec from data regions
Strongly-ordered	No cache, no buffer, sync (periph regs)	Device	No cache, buffered (AHB periph)
Normal cached	WBWA (Write-Back, Write-Alloc) — RAM	Shared	Set S-bit for inter-core shared SRAM
CORTEX-M7 PERFORMANCE FEATURES			
Branch prediction	2-level adaptive predictor	Speculative fetch	Pre-fetches target before branch resolves
Superscalar	Dual-issue: 2 instructions/cycle	Write buffer	8-entry store buffer (non-blocking)
FPU latency	DP FADD: 3cy, FMUL: 5cy, FDIV: 15cy	SIMD	UADD8, SADD16 parallel byte/halfword ops

ECC	Flash + TCM ECC (error correction)	Parity	AXI SRAM parity protection
NOTE: Place time-critical CM7 ISRs in ITCM (0x00000000). Place DMA buffers in non-cacheable SRAM (use <code>__attribute__((section('.dma_buffer')))</code>).			

20. EMBEDDED C — SYNTAX, PATTERNS & PROJECT STRUCTURE

EMBEDDED C DATA TYPES & QUALIFIERS			
stdint.h types	uint8_t, int16_t, uint32_t, int64_t	stdbool.h	bool, true, false
volatile	Prevents optimiser removing read	const	Read-only; stored in Flash if global
static (local)	Persists between function calls	static (global)	File-scope only (private to .c file)
register	Hint: use CPU register (mostly ignored)	restrict	Pointer aliasing hint for optimiser
__attribute__	GCC: aligned, packed, section, weak	inline	Request function inlining
STRUCTS, BITFIELDS & UNIONS			
Packed struct	typedef struct __attribute__((packed)) { uint8_t id; uint16_t val; uint8_t crc; } Packet_t;		
Bitfield (reg)	typedef struct { uint8_t EN:1; uint8_t MODE:2; uint8_t ERR:1; uint8_t :4; } RegCtrl_t;		
Union overlay	typedef union { uint32_t word; uint8_t bytes[4]; float f; } Word32_t;		
REGISTER-LEVEL GPIO (ARM/STM32)			
Set pin HIGH	GPIOA->BSRR = (1UL << 5); // Set PA5		
Set pin LOW	GPIOA->BSRR = (1UL << (5 + 16)); // Reset PA5		
Toggle pin	GPIOA->ODR ^= (1UL << 5); // Toggle PA5		
Read pin	if (GPIOA->IDR & (1UL << 0)) { } // Read PA0		
Config output	GPIOA->MODER = (1UL << (5*2)); // Mode=01 output		
INTERRUPT & ISR PATTERNS			
ISR prototype	void EXTI0_IRQHandler(void) { EXTI->PR1 = (1<<0); flag=1; }		
Shared flag	volatile uint8_t flag = 0; // in main: while(!flag){}; flag=0;		
Atomic read	__disable_irq(); val = shared; __enable_irq(); // critical section		
NVIC enable	HAL_NVIC_SetPriority(EXTIO_IRQn, 0, 0); HAL_NVIC_EnableIRQ(EXTIO_IRQn);		
STATE MACHINE PATTERN (Embedded)			
Enum states	typedef enum { ST_IDLE, ST_INIT, ST_RUN, ST_ERROR, ST_COUNT } State_t;		
SM loop	State_t state = ST_IDLE; while(1){ switch(state){ case ST_IDLE: ... break; } }		
USEFUL MACROS			
BIT macro	#define BIT(n) (1UL << (n))		
SET_BIT	#define SET_BIT(r,b) ((r) = BIT(b))		
CLR_BIT	#define CLR_BIT(r,b) ((r) &= ~BIT(b))		
TST_BIT	#define TST_BIT(r,b) (((r) >> (b)) & 1U)		
ARRAY_SIZE	#define ARRAY_SIZE(a) (sizeof(a)/sizeof((a)[0]))		
CLAMP	#define CLAMP(v,lo,hi) ((v)<(lo)?(lo):((v)>(hi)?(hi):(v)))		
MIN/MAX	#define MIN(a,b) ((a)<(b)?(a):(b)) #define MAX(a,b) ((a)>(b)?(a):(b))		
UNUSED param	#define UNUSED(x) (void)(x) // suppress compiler warning		
NOTE: Never use int/long for hardware registers — sizes vary by platform. Always use stdint.h fixed-width types.			

EMBEDDED C — PROFESSIONAL PROJECT STRUCTURE

EMBEDDED C PROJECT STRUCTURE — STM32 / Bare-metal / FreeRTOS	
MyProject/	<i>Root project folder</i>
└ Core/	<i>STM32CubeMX-generated core files</i>
└ Inc/	<i>main.h, gpio.h, usart.h, tim.h, stm32h7xx_hal_conf.h</i>
└ Src/	<i>main.c, gpio.c, usart.c, tim.c, stm32h7xx_it.c (ISRs)</i>
└ Startup/	<i>startup_stm32h755xx.s — reset handler, vector table</i>
└ Drivers/	<i>HAL + BSP drivers (do not edit)</i>
└ STM32H7xx_HAL_Driver/	<i>Cube HAL .c/.h (generated, version-controlled separately)</i>
└ CMSIS/	<i>ARM CMSIS core headers (core_cm7.h etc.)</i>
└ Middleware/	<i>Third-party middleware</i>
└ FreeRTOS/	<i>tasks.c, queue.c, timers.c, heap_4.c, FreeRTOSConfig.h</i>
└ LwIP/	<i>TCP/IP stack (optional — Ethernet apps)</i>
└ USB_Device/	<i>USB CDC/MSC/HID class stack</i>
└ App/	<i>YOUR application code (clean separation from HAL)</i>
└ Inc/	<i>app_config.h, sensor.h, comms.h, state_machine.h</i>
└ Src/	<i>app_main.c, sensor.c, comms.c, state_machine.c</i>
└ Tasks/	<i>FreeRTOS tasks: task_sensor.c, task_comms.c, task_ui.c</i>
└ Util/	<i>ring_buffer.c, crc.c, logger.c, assert.c</i>
└ CM4/	<i>Cortex-M4 sub-project (dual-core STM32H7)</i>
└ Core/	<i>main_cm4.c, ipc.c, hsem.c, cm4_tasks.c</i>
└ Startup/	<i>startup_stm32h755xx_CM4.s</i>
└ Tests/	<i>Unit tests (Unity / CMock framework)</i>
└ test_sensor.c	<i>Mocked HAL, test sensor driver in isolation</i>
└ test_crc.c	<i>Pure C — no hardware dependency</i>
└ Scripts/	<i>Build & deployment scripts</i>
└ flash.sh	<i>OpenOCD / STM32CubeProg flash script</i>
└ gdb_init.gdb	<i>GDB startup: break main, load, run</i>
└ generate_version.py	<i>Embeds git hash + date into firmware binary</i>
└ Docs/	<i>Schematics PDF, memory map, protocol specs</i>
└ .github/	<i>CI/CD: build.yml (gcc-arm, unit tests, size check)</i>
└ CMakeLists.txt	<i>CMake build (preferred over Makefile for larger projects)</i>
└ MyProject.ioc	<i>STM32CubeMX config (do not manually edit)</i>
└ STM32H755ZITX_FLASH.ld	<i>Linker script — MEMORY regions, SECTIONS</i>
└ .gitignore	<i>Ignore: build/, *.elf, *.o, *.bin, *.map</i>

LINKER SCRIPT — MEMORY REGIONS & SECTIONS	
MEMORY block	<code>MEMORY { FLASH(rx):ORIGIN=0x08000000,LENGTH=1024K DTCMRAM(rwx):ORIGIN=0x20000000,LENGTH=128K RAM(rwx):ORIGIN=0x24000000,LENGTH=512K }</code>
.text section	<code>.text : { *(.isr_vector) *(.text*) *(.rodata*) } > FLASH</code>
.data section	<code>.data : { *(.data*) } > DTCMRAM AT>FLASH /* LMA in Flash, VMA in RAM */</code>
.bss section	<code>.bss : { *(.bss*) *(COMMON) } > DTCMRAM /* Zero-init by startup */</code>
DMA buffer	<code>.dma_buf (NOLOAD): { *(.dma_buffer) } > RAM /* non-cached SRAM for DMA */</code>
ITCM section	<code>.itcmram : { *(.itcmram*) } > ITCMRAM AT>FLASH /* Copy ISR to fast TCM */</code>
C ATTRIBUTE PLACEMENT	
Place in ITCM	<code>__attribute__((section(".itcmram"))) void USART1_IRQHandler(void) { ... }</code>
DMA buffer	<code>__attribute__((section(".dma_buffer"), aligned(32))) uint8_t rxBuf[256];</code>

No init RAM	<code>__attribute__((section(".noinit"))) uint32_t resetCause; /* survives reset */</code>
FREERTOS TASK TEMPLATE	
Task create	<code>xTaskCreate(vSensorTask, "Sensor", 512, NULL, 3, &hSensor); // 512 words stack</code>
Task body	<code>void vSensorTask(void *arg){ for(;;){ xSemaphoreTake(sem,pdMS_TO_TICKS(100)); /* work */ }</code>
Queue send	<code>xQueueSend(xDataQ, &packet, pdMS_TO_TICKS(10)); // from ISR: xQueueSendFromISR()</code>
Delay	<code>vTaskDelayUntil(&xLast, pdMS_TO_TICKS(10)); // precise 10ms period (not vTaskDelay)</code>
NOTE: Use <code>vTaskDelayUntil()</code> not <code>vTaskDelay()</code> for periodic tasks — eliminates drift. Stack in WORDS (4 bytes each on ARM).	

22. INDUSTRIAL MEASUREMENT STRUCTS — Embedded C

Digital I/O Payload

```
// Digital I/O data payload structure
typedef struct {
    uint32_t input_states;        // 32-bit input state mask (bit0=DI0 ... bit31=DI31)
    uint32_t output_states;       // 32-bit output state mask (bit0=DO0 ... bit31=DO31)
    uint32_t input_change_mask;   // bits that changed since last poll
    uint32_t output_change_mask;  // output bits changed by last command
    uint8_t di_count;             // number of physical DI channels
    uint8_t do_count;             // number of physical DO channels
    uint8_t reserved[2];          // pad to 4-byte boundary
} digital_io_payload_t;
```

System Health Payload

```
// System health payload structure
typedef struct {
    uint8_t health_status;        // Overall health percentage (0-100)
    uint8_t cpu_usage;            // CPU usage percentage (0-100)
    uint8_t memory_usage;         // Memory usage percentage (0-100)
    uint8_t temperature;         // Internal temperature in °C
    uint16_t voltage_mv;          // Supply voltage in millivolts
    uint16_t error_count;         // Number of errors since last reset
    uint32_t uptime_s;            // Seconds since last power-on
    uint32_t last_error_code;     // Most recent error/fault code
} system_health_payload_t;
```

AC Voltage & Current Measurement Payload

```
// Single-phase AC measurement payload
typedef struct {
    uint32_t timestamp_ms;        // Measurement timestamp (ms since boot)
    int32_t voltage_mv;           // RMS voltage in millivolts
    int32_t current_ma;           // RMS current in milliamps
    int32_t active_power_mw;       // Active power P in milliwatts
    int32_t reactive_power_mvar;  // Reactive power Q in milli-VAR
    uint32_t apparent_power_mva;  // Apparent power S in milli-VA
    int16_t power_factor_x1000;   // PF × 1000 e.g. 0.95 → 950
    int16_t phase_angle_cdeg;     // Phase angle in centidegrees e.g. 18.5° → 1850
    uint16_t frequency_mhz;       // Frequency × 1000 e.g. 50.000Hz → 50000
    uint16_t thd_pct_x100;        // THD% × 100 e.g. 5.2% → 520
    uint8_t channel;              // 0=L1 1=L2 2=L3 3=Neutral
    uint8_t flags;                // bit0=OV bit1=UV bit2=OC bit3=PF low
    uint16_t crc16;               // CRC-16/Modbus over all preceding bytes
} ac_measurement_t;              // 32 bytes packed
```

Three-Phase Power Payload

```
// Three-phase power summary payload
typedef struct {
    ac_measurement_t phase[3];    // L1 [0], L2 [1], L3 [2] per-phase data
    int32_t total_active_w;        // P_total = P_L1 + P_L2 + P_L3 (watts)
    uint32_t total_apparent_va;    // S_total = sqrt3 × V_L × I_L (VA)
    int32_t total_reactive_var;    // Q_total = sqrt(S² - P²) (VAR)
    int16_t voltage_unbal_x100;    // Voltage unbalance factor × 100 (%)
    int16_t current_unbal_x100;    // Current unbalance factor × 100 (%)
    uint32_t meter_serial;         // Unique meter serial number
    uint8_t tariff_zone;           // 0=off-peak 1=standard 2=peak
    uint8_t reserved[3];           // alignment padding
} three_phase_payload_t;
```

Modbus RTU Frame & Float Union

```
// Modbus float union - dual view: two 16-bit registers OR one IEEE-754 float
typedef union {
    uint16_t reg[2];           // reg[0]=high word, reg[1]=low word (big-endian)
    uint32_t raw32;           // combined 32-bit raw value
    float value;               // IEEE-754 float (verify byte order with meter!)
} modbus_float_t;

// Modbus RTU request frame
typedef struct __attribute__((packed)) {
    uint8_t slave_addr;       // 1-247
    uint8_t function_code;    // 0x03=read regs 0x06=write reg 0x10=write multi
    uint16_t start_register;  // first register address (big-endian)
    uint16_t register_count;  // number of registers to read/write
    uint16_t crc16;           // CRC-16/Modbus (little-endian append)
} modbus_request_t;         // 8 bytes
```

Meter Status / Alarm Bitfield

```
// Meter alarm and status flags - maps directly to Modbus coil register
typedef struct {
    uint16_t overvoltage      : 1; // bit 0 - phase voltage > OV threshold
    uint16_t undervoltage     : 1; // bit 1 - phase voltage < UV threshold
    uint16_t overcurrent      : 1; // bit 2 - phase current > OC threshold
    uint16_t pf_low           : 1; // bit 3 - power factor < PF min setting
    uint16_t freq_high        : 1; // bit 4 - frequency > 50.5 Hz
    uint16_t freq_low         : 1; // bit 5 - frequency < 49.5 Hz
    uint16_t phase_loss       : 1; // bit 6 - one or more phases missing
    uint16_t earth_fault      : 1; // bit 7 - earth leakage detected
    uint16_t ct_open          : 1; // bit 8 - CT secondary open circuit
    uint16_t reverse_energy    : 1; // bit 9 - energy flow reversed (export)
    uint16_t tamper           : 1; // bit 10 - tamper event detected
    uint16_t comms_timeout    : 1; // bit 11 - master poll timeout
    uint16_t reserved         : 4; // bits 12-15 unused
} meter_status_t;
```

Industrial Sensor Packet Union (UART / CAN Transmission)

```
// Industrial sensor packet - dual view: structured fields OR raw byte array
typedef union {
    struct __attribute__((packed)) {
        uint8_t start_byte; // 0xAA - frame sync
        uint8_t device_id;  // 1-247 (Modbus slave address)
        uint8_t msg_type;    // 0x01=meas 0x02=alarm 0x03=config
        uint8_t length;      // payload byte count (excl. header+crc)
        ac_measurement_t payload; // 32-byte measurement body
        uint16_t crc;        // CRC-16 over [device_id .. payload]
    };
    uint8_t raw[38];        // entire frame as byte array for UART TX
} industrial_frame_t;

// Usage: uart_write(frame.raw, sizeof(industrial_frame_t));
```

Calibration Data (stored in EEPROM / Flash)

```
// Calibration coefficients - stored in Flash/EEPROM with CRC integrity check
typedef struct __attribute__((packed, aligned(4))) {
    uint32_t magic;          // 0xCAFEBADE - marks valid calibration block
    float v_gain[3];         // voltage gain correction per phase [L1,L2,L3]
    float v_offset_mv[3];    // voltage offset in mV per phase
    float i_gain[3];         // current gain correction per CT channel
    float i_offset_ma[3];    // current offset in mA per CT channel
    float phase_comp_deg[3]; // phase compensation angle in degrees
    uint32_t cal_timestamp;  // UNIX epoch of last calibration
    uint16_t hw_version;     // PCB revision e.g. 0x0102 = v1.2
    uint16_t fw_version;     // firmware version
}
```

```
uint32_t crc32;           // CRC-32 integrity check over all above bytes
} calibration_data_t;     // 80 bytes total

// Read rule: validate magic == 0xCAFEFABE AND crc32 before using any field
```

STRUCT / UNION QUICK REFERENCE			
__attribute__((packed))	Remove padding – exact byte layout	__attribute__((aligned(n)))	Force n-byte alignment (DMA need 32)
volatile member	uint32_t volatile reg – prevents optim.	const pointer	const sensor_t *p – read-only access
Padding rule	Member aligns to its own size by default	Zero padding	uint8_t reserved[N] – explicit pad
Union size	= size of largest member	Struct size	= sum of members + padding
memcpy rule	Always use memcpy() for packed structs	Never deref	Packed ptr deref = bus fault on M4/M7
Endian BE→LE swap 16	((v>>8)&0xFF) ((v&0xFF)<<8)	Endian BE→LE swap 32	__builtin_bswap32(v) or ntohl(v)
Magic number	0xCAFEFABE / 0xDEADBEEF – validity mark	CRC choice	CRC-16/Modbus for frames; CRC-32 for NVM
NOTE: Scale to integers in structs — never store raw floats in protocol frames (endian/NaN issues). Use float only in local processing or calibration NVM with CRC protection.			

23. AC VOLTAGE, POWER & POWER FACTOR

AC WAVEFORM FUNDAMENTALS			
Instantaneous v(t)	$v(t) = V_m \times \sin(\omega t + \phi)$	Instantaneous i(t)	$i(t) = I_m \times \sin(\omega t + \phi - \delta)$
Angular frequency ω	$\omega = 2\pi f$ [rad/s]	Period T	$T = 1/f$ (SA: $1/50 = 20$ ms)
Peak voltage V_m	$V_m = V_{rms} \times \sqrt{2}$	Peak current I_m	$I_m = I_{rms} \times \sqrt{2}$
RMS voltage	$V_{rms} = V_m / \sqrt{2}$ (sine only)	RMS general	$V_{rms} = \sqrt{(1/T \times \int v^2 dt)}$
Vrms (non-sine)	$V_{rms} = \sqrt{(V_1^2 + V_2^2 + V_3^2 + \dots)}$ (harmonics)	Form factor	$FF = V_{rms} / V_{avg} = 1.1107$ (sine)
Crest factor	$CF = V_{peak} / V_{rms} = \sqrt{2} \approx 1.414$ (sine)	SA nominal voltage	230 V RMS phase, 400 V L-L (NRS 048)
POWER TRIANGLE — P, Q, S			
Active power P	$P = V_{rms} \times I_{rms} \times \cos\phi$ [W]	Resistive load	$P = I^2 R = V^2 / R$
Reactive power Q	$Q = V_{rms} \times I_{rms} \times \sin\phi$ [VAR]	Inductive Q	$Q = I^2 \times X_L$ (positive, lagging)
Apparent power S	$S = V_{rms} \times I_{rms}$ [VA]	Power triangle	$S^2 = P^2 + Q^2$
Power factor PF	$PF = \cos\phi = P / S$	Phase angle ϕ	$\phi = \arctan(Q / P)$
Lagging PF	Current lags voltage – inductive load	Leading PF	Current leads voltage – capacitive
Unity PF	$\cos\phi = 1$ (pure resistive, $P = S$)	Zero PF	$\cos\phi = 0$ (pure reactive, $P = 0$)
THREE-PHASE POWER			
3ϕ Active P	$P = \sqrt{3} \times V_L \times I_L \times \cos\phi$ [W]	Also expressed as	$P = 3 \times V_{ph} \times I_{ph} \times \cos\phi$
3ϕ Reactive Q	$Q = \sqrt{3} \times V_L \times I_L \times \sin\phi$ [VAR]	3ϕ Apparent S	$S = \sqrt{3} \times V_L \times I_L$ [VA]
Star V_L–V_{ph}	$V_L = \sqrt{3} \times V_{ph}$ (SA: $400V = \sqrt{3} \times 230V$)	Star $I_L = I_{ph}$	Line current = phase current
Delta $V_L = V_{ph}$	Line voltage = phase voltage	Delta $I_L = \sqrt{3} \times I_{ph}$	$I_L = \sqrt{3} \times$ phase winding current
Symmetrical load	All 3 phases equal – neutral current = 0	Unbalanced load	Neutral carries difference current
POWER FACTOR — MEASUREMENT & CORRECTION			
Displacement PF	$DPF = \cos(\phi_1)$ – fundamental component only	True PF	$TPF = P / S$ (includes harmonics)
True PF with THD	$PF = DPF / \sqrt{(1 + THD_I^2)}$	THD effect	High THD_I reduces true PF even at $\phi=0$
PF from P & Q	$PF = P / \sqrt{(P^2 + Q^2)}$	PF from energy meter	$PF = W_h / VA_h$ (over same interval)
Leading / Lagging sign	IEEE: +Q = inductive (lagging)	IEC convention	IEC: +Q = inductive (same as IEEE)
Vars required for corr.	$Q_c = P \times (\tan\phi_1 - \tan\phi_2)$	Capacitor size	$C = Q_c / (\omega \times V^2)$ [F]
PF correction target	Eskom industrial target: $PF \geq 0.90$	Penalty zone	$PF < 0.90 \rightarrow$ Eskom reactive energy charge
POWER FACTOR CORRECTION (PFC)			
Passive PFC	Capacitor bank switched in steps	C bank step size	$Q_{step} = C \times \omega \times V^2$ (per capacitor)
Active PFC	Boost converter forces sinusoidal input I	Active PFC THD	$< 5\%$ input THD, $PF > 0.99$

Auto PF controller	Measures KVAR, switches C banks via relay	Hunting prevention	Deadband ± 0.02 PF around setpoint
Harmonic filter	Passive: tuned LC Active: inject anti-harmonic current	Resonance risk	Avoid C + transformer resonance
REACTIVE ENERGY & BILLING (SA Eskom NRS 048 / NRS 057)			
Active energy	kWh = P[kW] × hours (billed)	Reactive energy	kVARh = Q[kVAR] × hours (penalised)
Apparent energy	kVAh = S[kVA] × hours	Demand	kVA demand = avg S over 30-min window
MD (Max Demand)	Highest 30-min kVA average in billing month	MD charge	R/kVA/month on peak demand
NMD (Notified MD)	Agreed max demand with utility	Excess charge	Penalty for exceeding NMD
ToU (Time of Use)	Peak 07:00–10:00 & 18:00–20:00 weekdays	Off-peak	22:00–06:00 weekdays, full weekend
VOLTAGE & CURRENT QUALITY INDICATORS			
Voltage regulation VR	VR% = (VNL - VFL) / VFL × 100%	Good VR target	< 5% for feeders (NRS 048-2)
Voltage unbalance VUF	VUF% = Vneg / Vpos × 100%	Limit	< 2% sustained (IEC 61000-2-2)
Voltage sag	RMS drop 10–90%, duration 10ms–1min	Swell	RMS rise > 110%, < 1 min
Flicker Pst	Short-term perceptibility index (10-min)	Plt	Long-term (2-hour rolling cube mean)
Current unbalance	I_unbal% = I_max_dev / I_avg × 100%	Motor derating	1% V unbalance → ~6–10% I unbalance
NOTE: In SA: supply is 230V/400V 50Hz (NRS 048). Industrial PF penalty applies below 0.90. Use kVAR meters or smart meters with reactive energy registers. Always check both true PF and displacement PF on non-linear loads (VFDs, UPS, SMPS).			

24. BATTERY SOC — STATE OF CHARGE MEASUREMENTS

SOC DEFINITION & METHODS			
SOC definition	SOC = Q_remaining / Q Rated × 100 (%)	100% SOC	Fully charged cell
0% SOC	Fully discharged (Vmin cutoff reached)	Capacity Qn	Rated Ah e.g. 100 Ah @ C10 rate
C-rate	C/n = full discharge in n hours	1C example (100Ah)	100A discharge → empty in 1 hour
METHOD 1 — COULOMB COUNTING (Current Integration)			
Continuous form	SOC(t) = SOC0 + (η/Qn) × ∫0^t I(τ) dt	Sign convention	+I = charge, -I = discharge
Discrete (MCU)	SOC += (I × Δt × η) / (Qn × 3600) × 100	Units	I[A], Δt[s], Qn[Ah] → SOC[%]
Charge efficiency η	Li-ion: η ≈ 0.99 Lead-acid: η ≈ 0.85–0.90	Self-discharge	Subtract leakage current from I
Error accumulation	Drifts over time — needs periodic reset	Reset trigger	At full charge detection (Vmax + Itaper)
ADC requirement	12–16 bit ADC on shunt resistor	Shunt example	10mΩ shunt: 100A → 1.0V drop
METHOD 2 — OCV LOOKUP (Open Circuit Voltage)			
OCV-SOC relation	SOC = f(OCV) — lookup table per chemistry	Rest time needed	Li-ion: ≥ 2h rest; Lead-acid: ≥ 4h
Li-ion NMC OCV	100%: 4.20V 50%: 3.70V 0%: 3.00V	LFP OCV (flat)	100%: 3.60V 50%: 3.30V 0%: 2.80V

Lead-acid OCV	100%: 12.70V 50%: 12.20V 0%: 11.80V	NiMH OCV	100%: 1.45V 50%: 1.25V 0%: 1.10V
OCV error under load	IR drop distorts reading – not usable live	Temperature effect	OCV shifts $\approx -2\text{mV}/^{\circ}\text{C}$ (lead-acid)
METHOD 3 — KALMAN FILTER (Optimal Estimator)			
State vector	$\mathbf{x} = [\text{SOC}, \text{Vrc1}, \text{Vrc2}]^T$ (+ RC model states)	Measurement	$z = V_{\text{terminal}}$ (measured voltage)
Process model	$\mathbf{x}(k+1) = \mathbf{A}\mathbf{x}(k) + \mathbf{B}\mathbf{u}(k) + \mathbf{w}(k)$	Measurement model	$z(k) = \mathbf{H}\mathbf{x}(k) + v(k)$
EKF linearise	Jacobian of $h(\mathbf{x})$ at each time step	Advantage	Handles non-linear OCV-SOC curve
Typical accuracy	$\pm 1\text{--}3\%$ with good cell model + temperature	Compute cost	Feasible on Cortex-M4 @ 1Hz update
BATTERY CHEMISTRY — OCV & VOLTAGE RANGES			
Li-ion (NMC)	Nom 3.6V Full 4.20V Empty 3.00V/cell	Li-ion (LFP)	Nom 3.2V Full 3.60V Empty 2.80V
LiPo	Nom 3.7V Full 4.20V Storage 3.85V	Lead-acid (12V)	Full 12.7V 50% 12.2V Min 11.8V
Series string Vtotal	$V_{\text{total}} = n \times V_{\text{cell}}$	Parallel: same V	$I_{\text{total}} = n \times I_{\text{cell}}$
SOH & BMS THRESHOLDS			
SOH (capacity)	$\text{SOH} = Q_{\text{measured}} / Q_{\text{rated}} \times 100\%$	EOL threshold	$\text{SOH} < 80\%$ (IEC 62619 / ISO 6469)
Cell imbalance trigger	$\Delta V_{\text{cell}} > 50\text{ mV} \rightarrow$ start balancing	Passive balance	Bleed resistor on high cell (heat)
Active balancing	Energy transfer cell-to-cell (inductor/cap)	OVP cutoff	Li-ion: $> V_{\text{max}}$ per cell (4.25V NMC)
UVP cutoff	Li-ion: $< 3.00\text{V}$ LFP: $< 2.80\text{V}$ per cell	OTP cutoff	Charge cutoff $> 45^{\circ}\text{C}$ Disch $> 60^{\circ}\text{C}$
SOC accuracy target	$\pm 3\%$ automotive (ISO 26262)	Stationary systems	$\pm 5\%$ acceptable (NRS 097 / IEC 62619)
NOTE: Always combine coulomb counting + OCV reset for best accuracy. In SA solar/UPS: recalibrate SOC at each full-charge cycle. Never charge Li-ion below 0°C — lithium plating causes permanent damage.			

25. CORS — CROSS-ORIGIN RESOURCE SHARING

CORS FUNDAMENTALS (RFC 6454 + Fetch Standard)			
Same-origin policy	Browser blocks cross-origin requests by default	Origin =	scheme + host + port (all 3 must match)
Cross-origin trigger	Origin differs in ANY of scheme / host / port	CORS purpose	Server opts-in to allow specific origins
Who enforces CORS	Browser only — curl/Postman ignore it	Not a firewall	Server still receives the request always
REQUEST TYPES			
Simple request	GET/POST + no custom headers + safe content-type	No preflight	Browser sends request directly with Origin
Preflight request	Browser sends OPTIONS before actual request	Triggers when	PUT/DELETE/PATCH or custom headers used
Credentialed request	Includes cookies: fetch({credentials:'include'})	Server must	Return specific origin (no *) + Allow-Credentials: true
Preflight caching	Access-Control-Max-Age: 3600 (seconds)	Chrome max	7200s Firefox max: 86400s
REQUEST HEADERS (Browser → Server)			
Origin	Origin: https://app.example.com	Sent by	Browser — automatically on every cross-origin req.
Request-Method	Access-Control-Request-Method: PUT	In	OPTIONS preflight only
Request-Headers	Access-Control-Request-Headers: X-Auth-Token	In	OPTIONS preflight — lists custom headers
RESPONSE HEADERS (Server → Browser)			
Allow-Origin *	Access-Control-Allow-Origin: *	Restriction	Wildcard blocks credentials (cookies/auth)
Allow-Origin specific	Access-Control-Allow-Origin: https://app.com	Required when	credentials=true OR restricted access needed
Allow-Methods	Access-Control-Allow-Methods: GET, POST, PUT, DELETE, PATCH, OPTIONS	List	All methods the client may call
Allow-Headers	Access-Control-Allow-Headers: Content-Type, Authorization, X-Custom	List	Every custom header client will send
Allow-Credentials	Access-Control-Allow-Credentials: true	Critical	Cannot combine with Allow-Origin: *
Expose-Headers	Access-Control-Expose-Headers: X-RateLimit-Remaining	Purpose	Makes non-simple response headers readable in JS
Max-Age	Access-Control-Max-Age: 3600	Effect	Browser caches preflight — reduces OPTIONS calls
PREFLIGHT FLOW (4 STEPS)			
1 Browser→Server	OPTIONS /api/sensor Origin: https://dashboard.com Access-Control-Request-Method: PUT Access-Control-Request-Headers: Authorization		
2 Server→Browser	HTTP 200 OK Access-Control-Allow-Origin: https://dashboard.com Access-Control-Allow-Methods: GET,PUT Access-Control-Allow-Headers: Authorization Access-Control-Max-Age: 3600		
3 Browser→Server	PUT /api/sensor (actual request — proceeds only if Step 2 headers are valid) Origin: https://dashboard.com Authorization: Bearer <token>		
4 Server→Browser	HTTP 200 OK Access-Control-Allow-Origin: https://dashboard.com Content-Type: application/json { response body }		
SERVER CONFIGURATION EXAMPLES			
Node / Express	app.use(cors({ origin:'https://app.com', methods:['GET','POST','PUT','DELETE'], allowedHeaders:['Content-Type','Authorization'], credentials:true, maxAge:3600 })));		

Python FastAPI	<code>app.add_middleware(CORSMiddleware, allow_origins=['https://app.com'], allow_credentials=True, allow_methods=['*'], allow_headers=['*'], max_age=3600)</code>		
Nginx config	<code>add_header 'Access-Control-Allow-Origin' 'https://app.com' always; add_header 'Access-Control-Allow-Methods' 'GET,POST,PUT,DELETE,OPTIONS' always; add_header 'Access-Control-Max-Age' '3600' always;</code>		
ASP.NET Core	<code>builder.Services.AddCors(o=>o.AddPolicy("Policy", b=>b.WithOrigins("https://app.com").AllowAnyMethod().AllowAnyHeader().AllowCredentials()));</code>		
ESP32 / embedded	<code>httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", ""); // wildcard OK for local IoT with no cookies</code>		
COMMON CORS ERRORS & FIXES			
No Allow-Origin header	Server missing CORS headers entirely	Fix	Add CORS middleware / response headers on server
Wildcard + credentials	Allow-Origin:* with credentials:include	Fix	Replace * with exact origin string
Preflight 404 / 405	Server has no OPTIONS route	Fix	Add OPTIONS handler or auto-handle in middleware
Header not exposed	Custom response header unreadable in JS	Fix	Add to Access-Control-Expose-Headers
Mixed content	HTTPS page calling HTTP API	Fix	Both must use HTTPS – no exceptions in browsers
Cookie not sent	credentials:'include' missing in fetch()	Fix	Set credentials option + ensure SameSite=None;Secure
CORS IN EMBEDDED / IoT CONTEXT			
IoT HTTP server	ESP32/STM32-LwIP – wildcard OK without cookies	Reverse proxy	Nginx handles CORS – MCU never sees preflight
WebSocket CORS	WS upgrade is NOT subject to CORS headers	WS auth	Use token in URL param or first message payload
MQTT over WS	MQTT-WS follows WebSocket rules (not HTTP CORS)	Server-to-server	HTTP calls between servers bypass CORS entirely
NOTE: CORS is a browser security feature only — it is NOT an authentication mechanism. Always add JWT/OAuth2 auth in addition to CORS. Misconfigured CORS (overly permissive) can expose APIs to CSRF attacks.			

26. MQTT — MESSAGE QUEUING TELEMETRY TRANSPORT

MQTT FUNDAMENTALS (ISO/IEC 20922)			
Protocol	Pub/Sub over TCP/IP (not request/response)	Transport	TCP port 1883 (plain) 8883 (TLS)
Broker	Central message router (Mosquitto, HiveMQ, EMQX)	Client	Publisher OR Subscriber (or both)
Payload	Any format: JSON, binary, plain text (up to 256MB)	MQTT 3.1.1	Most widely deployed version
MQTT 5.0	Adds: reason codes, message expiry, shared subs, topic aliases	Overhead	Fixed header: 2-5 bytes only – IoT optimised
TOPICS & WILDCARDS			
Topic structure	Hierarchical: factory/line1/machine3/temp	Case sensitive	Temperature ≠ temperature
Single-level +	factory/+/machine3 matches factory/line1/machine3	Multi-level #	factory/# matches everything under factory/
\$ topics	\$\$SYS/# – broker system info (clients, messages, load)	Retained msg	Broker stores last message per topic – new subs get it immediately
Shared subs MQTT5	\$share/group/topic – load-balance across subscribers	Topic alias	Short int replaces long topic string (MQTT 5.0)
QUALITY OF SERVICE (QoS)			
QoS 0 — At most once	Fire-and-forget: no ACK, possible loss	Best for	High-freq sensor data, loss tolerable
QoS 1 — At least once	PUBACK required: may deliver duplicates	Best for	Commands where duplicates are handled
QoS 2 — Exactly once	4-way handshake: PUBREC→PUBREL→PUBCOMP	Best for	Financial/billing data, critical events
QoS overhead	QoS0: 0 bytes QoS1: +2 bytes PacketID QoS2: +multi round-trips	Choice	Use QoS 0 or 1 on constrained devices
CONNECT, KEEP-ALIVE & LAST WILL			
CONNECT params	clientID, cleanSession, keepAlive(sec), willTopic, willPayload, willQoS, willRetain, username, password		
Keep-alive	Client sends PINGREQ every keepAlive interval Broker expects within 1.5× keepAlive		
Last Will (LWT)	Broker publishes willTopic/willPayload if client disconnects unexpectedly – device offline alert		
Clean session 0	Broker persists subscriptions + QoS1/2 queue across reconnects – MQTT persistent session		
Clean session 1	Fresh start on every connect – no stored subs or queued messages		
MQTT PACKET TYPES			
CONNECT / CONNACK	Client→Broker Broker→Client (return code 0=ok)	PUBLISH	Either direction: topicLen+topic+[packetID]+payload
PUBACK (QoS1)	Broker→Client: message received	PUBREC/REL/COMP	QoS2 four-step handshake
SUBSCRIBE/SUBACK	Client requests topic filter + QoS	UNSUBSCRIBE	Remove topic filter subscription
PINGREQ/PINGRESP	Keep-alive heartbeat (no payload)	DISCONNECT	Graceful disconnect (LWT not sent)
SECURITY			
TLS (port 8883)	Mutual TLS: client cert + server cert validation	Username/pass	CONNECT: username + hashed password

ACL (access control)	Broker config: which client can pub/sub which topic	JWT via MQTT5	Enhanced auth in CONNECT (MQTT 5.0)
Payload encryption	End-to-end: encrypt JSON before publish (AES-GCM)	Client ID	Must be unique – duplicate kicks old connection
EMBEDDED MQTT (IoT CLIENT)			
Paho MQTT C	Eclipse Paho: embedded C client, minimal footprint	MQTT-SN	UDP-based variant for mesh/radio (no TCP needed)
ESP32 MQTT	esp-mqtt component: configurable QoS, TLS, auto-reconnect	FreeRTOS task	Dedicated MQTT task: subscribe, publish, keepalive
Broker memory	Each retained msg stored in broker RAM/disk	Max clients	Mosquitto: ~1000 concurrent; HiveMQ: 10M+
Embedded budget	Paho embedded: ~8KB Flash, ~2KB RAM for MQTTv3	Buffer size	Rx/Tx buffer ≥ max message size (256B-4KB typical)
TOPIC DESIGN — INDUSTRIAL IoT BEST PRACTICES			
Telemetry topic	facility/site/device_id/measurement e.g. plant1/line3/motor7/temperature		
Command topic	facility/site/device_id/cmd/action e.g. plant1/line3/motor7/cmd/setpoint		
Status topic	facility/site/device_id/status e.g. plant1/line3/motor7/status (retained=true)		
LWT topic	facility/site/device_id/availability payload: 'online'/'offline' (retained=true)		
NOTE: Avoid deep topic trees (>5 levels). Use retained messages for device status and last-known values. For SA industrial: Mosquitto + TLS on local edge server reduces cloud dependency and latency.			

20b. EMBEDDED C — VARIABLE STORAGE PLACEMENT

STORAGE CLASS & MEMORY SECTION PLACEMENT IN EMBEDDED C			
DEFAULT PLACEMENT RULES (GCC ARM)			
Initialised global	int x = 5; → .data section (copied Flash→RAM at startup)	Uninit global	int x; → .bss (zero-initialised in RAM)
const global	const int k = 9; → .rodata in Flash (stays in Flash)	Local var	int y; → stack (auto, no section)
static local	static int cnt; → .bss or .data (RAM, persists)	register	Compiler hint – usually in CPU register
EXPLICITLY PLACE IN INTERNAL FLASH (Read-only data)			
Constant table	const uint16_t sine_table[256] __attribute__((section(".rodata"))) = { 0, 201, ... };		
String in Flash	const char fw_version[] __attribute__((section(".rodata"))) = "v1.2.3-2024";		
Lookup in Flash	static const float cal_coeff[8] = {1.02f, 0.98f ...}; // const → always Flash		
PROGMEM (AVR)	const PROGMEM uint8_t table[] = {0x00, 0xFF}; // AVR Harvard: data in Flash (pgmspace.h)		
AVR read back	uint8_t val = pgm_read_byte(&table[i]); // must use pgm_read_* – not direct deref		
PLACE IN SPECIFIC SRAM REGION (Section attribute)			
DTCM (M7 fast)	__attribute__((section(".dtcm_data"))) uint32_t isr_buffer[64]; // 0-wait DTCM RAM		
No init RAM	__attribute__((section(".noinit"))) uint32_t reset_cause; // survives soft reset		
Backup SRAM	__attribute__((section(".backup_sram"))) RTC_backup_t rtc_data; // VBAT retained		
AXI SRAM	__attribute__((section(".axi_sram"))) uint8_t frame_buf[640*480]; // large buffer		
DMA-SAFE NON-CACHED BUFFERS (Cortex-M7)			
DMA buffer	__attribute__((section(".dma_buffer"), aligned(32))) uint8_t rx_buf[256];		
Cache-off via MPU	Configure MPU region as Normal Non-Cacheable for .dma_buffer section		

Or use SRAM4	// SRAM4 is always non-cacheable on STM32H7 – map DMA buffers here		
Cache flush	SCB_CleanDCache_by_Addr((uint32_t*)tx_buf, sizeof(tx_buf)); // before DMA write		
Cache invalidate	SCB_InvalidateDCache_by_Addr((uint32_t*)rx_buf, sizeof(rx_buf)); // after DMA read		
EXTERNAL MEMORY (QSPI Flash, Parallel SRAM, SDRAM)			
QSPI Flash XIP	// Map QSPI to 0x90000000 (memory-mapped) then const array lives there automatically		
Ext. SRAM section	__attribute__((section(".ext_sram"))) uint8_t large_buf[1024*1024]; // 1MB ext. SRAM		
Linker MEMORY	EXTRAM (rwx) : ORIGIN = 0x60000000, LENGTH = 8M // FMC Bank1 SRAM		
Ext SDRAM section	__attribute__((section(".sdram"))) uint32_t video_buf[800*480]; // frame buffer in SDRAM		
EEPROM emulation	// Do NOT store structs directly – use HAL_FLASH_EE API: EE_WriteVariable(addr, val);		
VOLATILE — CRITICAL FOR SHARED VARIABLES			
ISR shared flag	volatile uint8_t data_ready = 0; // prevents optimizer removing ISR-written reads		
HW register ptr	volatile uint32_t * const GPIOA_ODR = (uint32_t*)0x48000014; // MMIO register		
Atomic 32-bit	__disable_irq(); shared_val = new_val; __enable_irq(); // Cortex-M atomic 32-bit		
LINKER SCRIPT SUMMARY (Memory Placement)			
.text	Code + const – stays in FLASH	.rodata	Read-only data – stays in FLASH
.data	Init'd globals – copied Flash→RAM at boot	.bss	Uninit'd globals – zeroed in RAM at boot
.noinit	Not touched by startup – retains value across reset	.dma_buffer	Non-cached RAM region – safe for DMA
.itcmram	Code copied to ITCM – zero-wait-state execution	.dtcm_data	Data in DTCM – fastest M7 data access
NOTE: Key rule: <i>const</i> = Flash. <i>volatile</i> = RAM, not optimised. DMA buffers = non-cached RAM, 32-byte aligned (Cortex-M7 cache line). Never put DMA buffers in cached SRAM without explicit cache flush/invalidate.			

MICROCHIP MPLAB X — PROJECT STRUCTURE

MICROCHIP MPLAB X IDE — PIC / dsPIC / PIC32 / SAM Project Structure	
MyMPLAB_Project.X/	MPLAB X project root (contains .X folder)
└─ Makefile	Auto-generated by MPLAB X — do not edit manually
└─ nbproject/	MPLAB X IDE project metadata (XML)
└─ project.xml	Project config: tool, device, compiler settings
└─ configurations.xml	Build configurations: debug / release / production
└─ Makefile-*.mk	Auto-generated makefiles per config
└─ main.c	Application entry point — main() + init calls
└─ Header Files/	Logical MPLAB folder (maps to physical dirs)
└─ mcc_generated_files/	MCC (MPLAB Code Configurator) generated headers
└─ app/	Application headers: sensor.h, comms.h, fsm.h
└─ Source Files/	Logical MPLAB folder
└─ mcc_generated_files/	MCC peripherals: uart.c, spi.c, tmr1.c, adc.c
└─ uart1.c / uart1.h	MCC UART driver (do not manually edit)
└─ tmr1.c / tmr1.h	MCC Timer1 driver
└─ spi1.c / spi1.h	MCC SPI driver
└─ system.c / system.h	MCC clock + oscillator config
└─ app/	YOUR application code
└─ sensor.c	Sensor read, scale, filter
└─ comms.c	Modbus RTU / MQTT / protocol handler

		state_machine.c	Application FSM
		util.c	Ring buffer, CRC, logging helpers
		Important Files/	Linker script + config bits
		p33EP512GP806.gld	GCC linker script for target device
		configuration_bits.c	#pragma config — oscillator, WDT, code protect
		Libraries/	Precompiled libs (.a) or MPLAB Harmony v3
		Harmony3/	MPLAB Harmony v3 framework (PIC32 / SAM devices)
		bsp/	Board Support Package — pinout, clock init
		config/	Harmony configurator output (device.h, system)
		peripheral/	Harmony PLIB: uart, spi, i2c, tc, adc, dmac
		thirdparty/	FreeRTOS / lwIP / TinyUSB (via Harmony package)
		Linker Files/	Custom linker script overrides
		debug/	Build output — ELF, HEX, MAP files
		MyProject.X.production.hex	HEX file for programming
		.gitignore	Ignore: debug/, *.d, *.p1, build/, dist/

MPLAB X KEY CONCEPTS & #pragma CONFIG			
CONFIGURATION BITS (#pragma config) — PIC16/18/dsPIC			
PIC18 example	#pragma config FOSC=HSMP, PLLCFG=ON, PRICLKEN=ON, WDTEN=OFF, MCLR=EXTMCLR, LVP=OFF		
dsPIC33 example	#pragma config FNOSC=FRCPPLL, IESO=OFF, FWDTEN=OFF, WINDIS=OFF, JTAGEN=OFF		
PIC32 example	#pragma config FPLLIDIV=DIV_2, FPLLMUL=MUL_20, FBDIV=DIV_1, FWDTEN=OFF, POSCMOD=HS		
MCC (MPLAB CODE CONFIGURATOR) WORKFLOW			
MCC purpose	GUI peripheral config → auto-generates C driver code	MCC3 vs MCC5	MCC5 (Melody) is newer – separate plugin
Regenerate	After MCC changes: Production > Generate Code	Custom code	Add user code only in USER CODE BEGIN/END blocks
MCC Melody	Component-based: drag-and-drop peripherals to canvas	Config file	MCC saves to .mc3 or project config JSON
PROGRAMMER / DEBUGGER TOOLS			
PICKit 4	USB programmer/debugger, 1.8-5V, ICSP/JTAG/SWD	SNAP	Low-cost debugger, 3.3V only, ICSP/JTAG
ICD 4	High-speed production debugger, 1.2-5.5V, all interfaces	PM4	Production programmer – gang programming support
ICSP pins	MCLR + PGD + PGC (+ VDD + GND = 5 wires)	Pinout	6-pin RJ-12 or 8-pin MicroStick connector
BUILD CONFIGURATIONS			
Debug build	No optimisation (-O0), debug symbols, fill erased areas	Release	-O2 optimisation, stripped symbols, smaller HEX
Production	Code protect fuses enabled, max optimisation	Memory summary	MPLAB shows Program / Data / Config memory usage %
NOTE: Never edit mcc_generated_files/ directly — MCC will overwrite on next generation. Place all custom code in app/ folder. Use #pragma config in a dedicated config_bits.c to keep it separate from main.c.			

BATTERY SOH, INTERNAL RESISTANCE & ADVANCED PARAMETERS			
STATE OF HEALTH (SOH)			
SOH (capacity)	$SOH_C = Q_{\text{measured}} / Q_{\text{rated}} \times 100\%$	Measurement	Full charge → discharge to Vmin → measure Ah
SOH (resistance)	$SOH_R = R_{i_new} / R_{i_aged} \times 100\%$ (or inverse)	EOL threshold	SOH < 80% capacity (IEC 62619, ISO 6469)

SOH vs cycles	NMC: ~80% SOH after 500 full cycles	LFP longevity	LFP: ~80% SOH after 2000-4000 cycles
Calendar ageing	SOH_loss = a × √(time) × exp(Ea/RT)	Arrhenius Ea	Higher temp → exponentially faster ageing
Degradation modes	Lithium plating SEI growth Active material loss	SEI layer	Solid Electrolyte Interphase forms on anode
INTERNAL RESISTANCE (Ri) MEASUREMENT			
DC Internal Resistance	DCIR = ΔV / ΔI (voltage step / current pulse)	Pulse test	Apply C/2 pulse for 10s, measure ΔV at 1s and 10s
DCIR formula	Ri = (VOC - Vload) / I_load [Ω]	Typical Li-ion	Ri: 5-50 mΩ/cell (aged = 2-3× new)
AC Impedance (EIS)	Electrochemical Impedance Spectroscopy – frequency sweep	EIS output	Nyquist plot: Ri + Rct + Warburg diffusion
Ohmic resistance R0	Instantaneous voltage drop on current step	Polarisation	Slower RC response: charge transfer + diffusion
Temperature effect	Ri doubles for every ~10°C drop below 25°C	Cold start	EV batteries pre-heat to 15°C before fast charge
Ri vs SOC	Ri increases at both extremes (SOC < 10% or > 90%)	Ri vs SOH	Ri increases as SOH decreases – aging indicator
EQUIVALENT CIRCUIT MODEL (ECM)			
1RC Thevenin	Vt = OCV(SOC) - I×R0 - I×R1×(1-e^(-t/τ1)) where τ1 = R1×C1		
2RC model	Vt = OCV - I×R0 - I×R1(1-e^-t/τ1) - I×R2(1-e^-t/τ2) [higher accuracy]		
Parameter ID	Fit R0, R1, C1, R2, C2 from HPPC pulse test data (Least Squares / MATLAB)		
STATE OF POWER (SOP) & POWER CAPABILITY			
SOP definition	Max continuous charge/discharge power at current state	Peak power	Ppeak = (OCV - Vmin)² / (4 × Ri) [W]
Charge power limit	P_chg = (Vmax - OCV) × I_chg_max	Discharge limit	P_dch = (OCV - Vmin) × I_dch_max
DCFC (fast charge)	Constant current to Vmax then constant voltage to Imin	1C to 80% SOC	NMC: ~45min LFP: ~40min at 1C
THERMAL MODEL			
Heat generation Q	Q = I² × Ri + T × ΔS × I (ohmic + entropic)	Thermal runaway	Exothermic reactions above 80°C (NMC)
Thermal resistance θ	ΔT = Q_watts × θ [°C/W]	Cell cooling	Liquid cooling: θ ≈ 0.5°C/W Air: 5-20°C/W
Safe temp range	Discharge: -20°C to 60°C Charge: 0°C to 45°C	Storage temp	15-25°C optimal for long shelf life
BMS PROTECTION PARAMETERS			
OVP (overvoltage)	NMC cutoff > 4.25V/cell LFP > 3.65V	UVP (undervolt)	NMC < 2.80V LFP < 2.50V per cell
OCP (overcurrent)	Hardware comparator: < 1µs response	OTP (overtemp)	Charge OTP: 45°C Discharge OTP: 60°C
Balancing trigger	ΔVcell > 30-50mV → activate balancing	Passive bal.	Bleed excess via 50-200mA resistor (heat)
Active balancing	Inductor or cap transfers energy between cells	Balance rate	Active: 1-5A Passive: 50-200mA
Pre-charge	Limit inrush to deeply discharged cell < C/10	Recovery	Apply 0.1C until Vcell > 3.0V (NMC) then normal
COULOMBIC EFFICIENCY & ENERGY EFFICIENCY			
Coulombic eff. CE	CE = Q_out / Q_in × 100%	Li-ion CE	99.5-99.9% per cycle (near unity at room temp)
Energy efficiency	η_E = E_out / E_in = CE × Vdch_avg / Vchg_avg × 100%	Typical	Li-ion: 92-97% round-trip energy efficiency

Lead-acid efficiency	CE: 85–90% Energy: 70–80%	NiMH	CE: 66–92% Energy: 65–70%
NOTE: Measure DCIR monthly in battery management applications — rising Ri is the earliest SOH degradation indicator. Never charge NMC below 0°C. Always log min/max Vcell, max Tcell, max Icell per cycle for warranty tracking.			

QUICK REFERENCE CONSTANTS & UNITS

PHYSICAL CONSTANTS & UNIT CONVERSIONS			
Boltzmann k	$1.38 \times 10^{-23} \text{ J/K}$	Electron charge q	$1.6 \times 10^{-19} \text{ C}$
Permittivity ϵ_0	$8.85 \times 10^{-12} \text{ F/m}$	Permeability μ_0	$4\pi \times 10^{-7} \text{ H/m}$
Speed of light c	$3 \times 10^8 \text{ m/s}$	Planck h	$6.626 \times 10^{-34} \text{ J}\cdot\text{s}$
1 kWh	$3.6 \times 10^6 \text{ J}$	1 Horsepower	746 W
dB Power	$10 \times \log_{10}(P2/P1)$	dB Voltage	$20 \times \log_{10}(V2/V1)$
Prefix p (pico)	10^{-12}	Prefix n (nano)	10^{-9}
Prefix μ (micro)	10^{-6}	Prefix m (milli)	10^{-3}
Prefix k (kilo)	10^3	Prefix M (Mega)	10^6
Prefix G (Giga)	10^9	Prefix T (Tera)	10^{12}

Comprehensive EE & ECE Formula Cheat Sheet • 30 Sections • Diodes | BJT | MOSFET | SCR | Op-Amp | Filters | Digital | Analogue | Embedded | MCU | Memory | Industrial Comms | DC Metering | AC Metering | WS | REST | PoE | Memory Map | Bitwise | CM7 | Embedded C